



Operations Research

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Two-Stage Learning to Branch in Branch-Price-and-Cut Algorithms for Solving Vehicle Routing Problems Exactly

Zhengzhong You, Yu Yang, Xinshang Wang, Wotao Yin

To cite this article:

Zhengzhong You, Yu Yang, Xinshang Wang, Wotao Yin (2026) Two-Stage Learning to Branch in Branch-Price-and-Cut Algorithms for Solving Vehicle Routing Problems Exactly. Operations Research

Published online in Articles in Advance 24 Feb 2026

. <https://doi.org/10.1287/opre.2023.0615>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

Copyright © 2026, INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes. For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

Contextual Areas

Two-Stage Learning to Branch in Branch-Price-and-Cut Algorithms for Solving Vehicle Routing Problems Exactly

Zhengzhong You,^{a,b} Yu Yang,^{a,b,*} Xinshang Wang,^c Wotao Yin^c

^aDepartment of Industrial and Systems Engineering, University of Florida, Gainesville 32611, Florida; ^bCenter for Applied Optimization, University of Florida, Gainesville 32611, Florida; ^cAlibaba Group US, Bellevue 98004, Washington

*Corresponding author

Contact: you.z@ufl.edu,  <https://orcid.org/0009-0007-3480-5578> (ZY); yu.yang@ise.ufl.edu,  <https://orcid.org/0000-0002-0502-7603> (YY); xinshang.w@alibaba-inc.com,  <https://orcid.org/0000-0003-4683-7167> (XW); wotao.yin@alibaba-inc.com,  <https://orcid.org/0000-0001-6697-9731> (WY)

Received: November 8, 2023

Revised: January 5, 2025; May 11, 2025

Accepted: January 14, 2026

Published Online in Articles in Advance:
February 24, 2026

Area of Review: Transportation

<https://doi.org/10.1287/opre.2023.0615>

Copyright: © 2026 INFORMS

Abstract. Branching is one of the most important components in branch-price-and-cut (BPC) algorithms for solving vehicle routing problems (VRPs) exactly. However, learning to branch is much more challenging in BPC than in branch-and-cut algorithms that are used for solving general mixed integer programs because branching, in this case, is generally performed by adding a dense constraint to the restricted master problem (RMP), and meanwhile, the variables in the RMP change constantly. To address such challenges, we propose the first effective *learning-to-branch framework* in BPC algorithms, leading to a novel *two-stage learning-based branching* (2LBB) strategy. This serves as an innovative learning-based enhancement for the cutting-edge three-phase branching strategy for column-generation-based algorithms. In the 2LBB, the first stage focuses on narrowing down the list of promising candidates using computationally cheap features thereby lessening dependence on linear programming testing. The second stage, meanwhile, diminishes the burden on heuristic testing through an innovative partial testing approach. Moreover, we propose a novel theoretical model characterizing the fundamental tradeoff between time spent making a single branching decision and the resulting branching quality. A formula, derived from the model, for dynamically adjusting the number of candidates to select for the second stage achieves consistently superior performance to ones obtained from trial-and-error tuning. The derivation easily generalizes to most branching strategies requiring time-consuming score computation. Through an extensive numerical study, we demonstrate that a dynamic version of the 2LBB, denoted by 2LBB-dy, achieves approximately 45% and 50% time reduction, respectively, compared with the state-of-the-art (SOTA) hand-crafted branching strategy in solving the capacitated vehicle routing problem (CVRP) and vehicle routing problem with time windows. In addition, our RouteOpt (an exact VRP solver available at <https://github.com/Zhengzhong-You/RouteOpt>), when equipped with the 2LBB-dy, achieves a 47% time reduction compared with the SOTA VRPSolver for the CVRP.

Funding: This work was supported by the National Science Foundation [Grant CMMI-2309667] and the Alibaba Group US [Grant AGR00022274].

Supplemental Material: This article includes an online appendix, computer code and data supporting the study's findings, and replication files. All supplemental materials, including the code, data, and files required to reproduce the results, are available at <https://doi.org/10.1287/opre.2023.0615>.

Supplemental Review: The empirical results in this paper were replicated. The code, data, and files required to reproduce the results were reviewed and are available at <https://doi.org/10.1287/opre.2023.0615>.

Keywords: learning to branch • column generation • exact methods • vehicle routing

1. Introduction

The vehicle routing problem (VRP) is a well-known discrete optimization problem that has been extensively studied in the operations research and management science community since its inception in 1959 (Dantzig and Ramser 1959). It seeks routes of the minimum cost (usually measured by the total travel

distance) to serve a set of consumers, subject to restrictions such as vehicle capacity in the capacitated VRP (CVRP) and time windows in the VRP with time windows (VRPTW). Optimizing the routes can also decrease waiting time and improve customer satisfaction, leading to wide applications in the industry such as e-commerce (Vinsensius et al. 2020), healthcare

(Heching et al. 2019), and emergency response (Zheng 2018), where the timely delivery of products and services is crucial.

Nonetheless, VRPs are generally challenging to solve optimally because of their combinatorial nature or, more intrinsically, the so-called NP-hardness. Approaches to solving VRPs exactly can generally be categorized into two types: *branch-and-cut* (B&C) algorithms solving compact (in terms of variables) formulations such as vehicle or commodity flow-based formulations (Bektaş et al. 2011), and *branch-price-and-cut* (BPC) algorithms solving the set-partitioning formulation (SPF; Costa et al. 2019). Branching is an indispensable component of these methods.

It is well known that the choice of branching variable significantly impacts the effectiveness of B&C algorithms for solving general mixed integer programs (MIPs; Achterberg and Wunderling 2013). Expectedly, branching remains one of the most important decisions in BPC algorithms for solving VRPs. In particular, our numerical experiments show that selecting the branching variable randomly increases computation time by more than eight times compared with the three-phase branching (3PB) strategy employed in the state-of-the-art VRP solver, the VRPSolver developed in Pessoa et al. (2020). Moreover, in contrast to pricing and cutting planes, which are usually customized based on specific problem structures for acceleration (Lysgaard et al. 2004, Lübbecke and Desrosiers 2005), branching does not rely on such information as much and tends to bear better generalization across a wide range of VRP variants. Therefore, the study of branching strategy holds significant potential in enhancing BPC methods for various vehicle routing-related problems.

To enhance the readability of this paper, the meanings of some key abbreviations are summarized in Table 1, sorted in order of decreasing frequency of use.

1.1. Motivation

Strong branching (SB; Applegate et al. 1995) is believed to be the most reliable way to obtain a small branch-and-bound tree (BBT; Costa et al. 2019). It considers a set of variables required to be integral but currently taking fractional values, hereafter referred to as *candidates*, computes the gains from branching on each of

them, and selects one with the highest gains to branch on (see Section 3.2 for more details). Despite its high efficacy in producing small-sized BBTs, SB can be extremely time-consuming because column generation (CG) is required to solve the child linear programming (LP) problems for all the candidates. To alleviate the computational burden, a hierarchical branching scheme is often adopted. In particular, the VRPSolver employs a 3PB strategy modified from Pecin et al. (2017b).

The 3PB, recognized as state-of-the-art (SOTA) branching strategy in BPC algorithms for solving VRPs, progressively narrows branching candidates through three phases: initial screening using pseudocost and fractionality, subsequent LP testing shortening the list, and finally heuristic testing selecting the candidate with the highest heuristic score. By gradually applying more computationally intensive evaluations to fewer candidates, 3PB effectively balances computational cost and decision quality, making it the leading branching strategy for BPC algorithms prior to this work. Its well-structured phased approach inspires the design of our two-stage learning-based branching (2LBB) and lays the foundation for making learning-accelerated branching strategies practical.

Although considered highly efficient, 3PB has two major weaknesses. Firstly, the screening process's *complete* reliance on pseudocost and fractionality before LP testing can potentially miss good candidates. Secondly, it requires performing *complete* time-consuming testing, whereas our newly proposed partial testing demonstrates that 80% of the time spent on complete heuristic testing is unnecessary. Overall, this research seeks to overcome these weaknesses to significantly reduce the computational effort while maintaining, if not improving, the branching decision quality.

Recently, considerable research effort has been dedicated to leveraging advanced machine learning (ML) techniques to mimic SB in the context of solving general MIPs and has successfully demonstrated that learned strategies can beat best-performing ones currently employed in solvers such as SCIP and CPLEX (Khalil et al. 2016, Alvarez et al. 2017, Yang et al. 2022). These ML models are trained with readily available historical and current-node information to solve compact formulations (i.e., formulations with a nonexponential and usually fixed number of variables). Once

Table 1. List of Abbreviations and Meanings

Abbreviation	Meaning	Abbreviation	Meaning
3PB	Three-phase branching	2LBB	Two-stage learning-based branching
BPC	Branch-price-and-cut	CG	Column generation
SB	Strong branching	BKF	Best k formula
BBT	Branch-and-bound tree	RMP	Restricted master problem
BBN	Branch-and-bound node	B&C	Branch-and-cut
SPF	Set-partitioning formulation	SOTA	State-of-the-art

trained, the ML models can select high-quality branching variables with little extra computation cost, achieving significant acceleration overall.

Nonetheless, models designed for branching selection in general MIPs mostly rely on three categories of features: static ones (e.g., initial LP matrix coefficients), dynamic ones from the current branch-and-bound node (BBN), and dynamic history-dependent ones (Alvarez et al. 2017, Yang et al. 2022). These approaches fail for BPC algorithms solving noncompact formulations, for example, SPF. To the best of our knowledge, no successful attempt to improve branching in BPC via ML has been accomplished before this study. The unfortunate ineffectiveness of existing learning-to-branch paradigms and the considerable difficulty in making them work for BPC are mainly because of the following three reasons.

First, to balance the resultant BBT, branching is typically not performed on a nominal variable in the restricted master problem (RMP) but rather achieved by adding a dense constraint (see more explanation in Section 3.2), after which, repetitive CG is needed to compute the exact LP value of a child BBN. For clarity, we refer to the LP at the current BBN as the *parent LP*, the one at a child BBN obtained from the parent LP by adding the branching constraint as the *starting child LP*, and the one resulting from solving the starting child LP with CG as the *ending child LP*. This branching constraint, together with the variables that have yet to be generated in the repetitive CG, causes the ending child LP to significantly deviate from the parent LP. Worse still, such deviation is not known until the starting child LP is solved with CG, making it extremely difficult, if not impossible, to accurately predict the LP value changes based on conventional features of the current BBN. Second, generated by CG iteratively, variables are added to the RMP over time. As a result, minimal historical information about the recently generated variables is available to aid the branching variable selection, which, unfortunately, is essential for existing ML models in the literature to make accurate predictions. Third, the accumulated effect of the first reason leads to acute changes to the RMP, where the current RMP can be almost completely different from the initial RMP at the root. Thus, the standard static features lose predictive power. In summary, these three reasons bring about unique obstacles to using dynamic current-node, dynamic history-dependent, and static features, accordingly, for learning to branch in BPC algorithms.

To overcome these challenges, new features that capture the underlying problem structure, rather than focusing solely on the RMP, are needed. Although these new features can be useful, their effectiveness depends on appropriate learning methods and workflows. We believe the newly proposed two-stage scheme addresses these needs. In the first stage, graph-based

features that are computationally cheap are used to narrow down the promising candidates where the learning target is the ranking of SB scores, thus reducing the dependence on LP testing. In the second stage, more computationally intensive features derived from LP testing are employed. At this stage, the learning target shifts from SB score ranking to the ratio of optimal value changes between the starting and ending child LPs relative to that of the parent LP. By applying a novel partial testing (see Section 4.4) that integrates predictions from the second stage ML model with LP scores, we efficiently determine which starting child LP should be further tested, enabling us to identify the best candidate with much less computational effort.

Moreover, SB, 3PB, and 2LBB necessitate testing multiple branching candidates before finalizing the decision. However, determining the optimal number of candidates to evaluate for minimizing overall solution time remains challenging as there is no established method to balance the tradeoff between the time spent on making a single branching decision and the resulting decision quality. To shed light on this fundamental tradeoff, we propose a novel theoretical model for deriving *Best K Formula* (BKF), which dynamically decides the best number of candidates to test based on the solution's progress. By embedding the distribution of local branching improvements, the BKF optimizes this tradeoff, leading to additional improvement in computational efficiency. It is important to emphasize that 2LBB is specifically designed for algorithms that incorporate CG. In contrast, the BKF is broadly applicable to branching strategies involving computationally intensive testing, including both BPC algorithms (the focus of this study) and B&C algorithms that do not require CG, as supported by our experiments in Section EC.8.4 of the e-companion.

In this paper, we develop a novel learning-to-branch framework with a theory-based dynamic adjustment scheme that can overcome the weaknesses of the current state-of-the-art branching strategy in BPC, namely 3PB. Although this study focuses on VRPs, we believe that most ideas introduced herein are easily transferable to BPC for addressing operational challenges in supply chain operations, healthcare, and the airline industry, for example, inventory routing problems (Engineer et al. 2012, Desaulniers et al. 2016), healthcare planning problems (Trautsamwieser and Hirsch 2014, Qiu et al. 2022), and crew rostering problems (Quesnel et al. 2020, Breugem et al. 2022).

1.2. Major Contributions and Outline

This paper concerns improving the branching decisions in BPC algorithms for solving VRPs exactly. It makes the following major contributions.

- We propose the first effective learning-to-branch framework in BPC algorithms. The first stage relies on

graph-based features that are computationally cheap to narrow down the promising candidates, whereas the second stage finalizes the decision with substantially less computation effort by utilizing features obtained from LP testing. This design ensures that the framework makes branching decisions fast and effectively selects high-quality candidates to generate a small-sized BBT. Furthermore, by integrating richer information at each stage and defining appropriate learning tasks, it advances both efficiency and effectiveness beyond 3PB, making it a SOTA branching method for BPC algorithms.

- We establish a novel theoretical model that gives rise to a formula for dynamically adjusting a key parameter in 2LBB and 3PB. The model characterizes the tradeoff between the testing time and branching decision quality, deciding the number of candidates to evaluate. It also generalizes to branching strategies reliant on computationally intensive scores and explains the performance differences between 2LBB and 3PB for CVRP and VRPTW instances.

- We integrate the 2LBB method into RouteOpt and perform extensive numerical experiments to support the design of the two-stage learning workflow. Our 2LBB reduces the average computation times by more than 45% compared with the SOTA 3PB method and generalizes well across varying instance sizes by consistently lowering CPU time and BBT size. When embedded into RouteOpt together with other effective techniques (see Section EC.5 of the e-companion), 2LBB achieves a 47% time reduction on benchmark instances compared with the VRPSolver and allows RouteOpt to prove optimality for an open CVRP instance.

The rest of the paper is organized as follows. In Section 2, we review related literature on branching in BPC algorithms and existing learning-to-branch methods. In Section 3, we include some preliminaries to facilitate readability. We describe our learning-to-branch framework in Section 4 and detail the learning methodology employed in the framework in Section 4.5. A theoretical model providing insights into the underlying tradeoffs is presented in Section 5. The effectiveness of the proposed framework is demonstrated via an extensive numerical study in Section 6. Lastly, in Section 7, we make some concluding remarks and identify future research directions. Details of selected proofs and numerical results are included in the e-companion.

2. Literature Review

Numerous VRP variants have been extensively investigated in the past 6.5 decades, leading to many effective exact algorithms. Laporte (2009) provides a thorough review of this topic. Currently, the best-performing ones are BPC algorithms, solving CVRP instances with up to 856 customers (Lima et al. 2014, Pessoa et al. 2020). This paper focuses on learning to branch in BPC algorithms.

We first review branching strategies employed in BPC algorithms from two perspectives: branching variable type and selection. Then, we discuss some most related ML attempts.

2.1. Branching Variable Type

Branching on arcs (or edges in the undirected case) is typically the default approach in B&C methods for solving the two-index vehicle flow formulation (Laporte and Nobert 1983). This prevalence is because the variables in such a flow formulation explicitly correspond to arcs/edges, and such branching selection leads to the most common branching on a single nominal variable in the formulation. Desrosiers et al. (1984) introduced branching on arcs in a BPC method for solving the VRPTW, which has become the primary choice in BPC algorithms (Irnich et al. 2010, Pessoa et al. 2020). Costa et al. (2019) underscored the popularity and highlighted its simplicity, ease of implementation, and robustness. In light of these considerations, we also adopt branching on edges in our learning-to-branch framework.

Despite the widespread application of branching on edges, Augerat et al. (1995) observed that this approach, for the CVRP, poses a relatively localized influence that might result in undesirably small changes in the dual bound after branching. Thus, they proposed to branch on sets, which entails selecting a set of vertices. If the total flow, induced by the current solution, passing through this set is fractional, then branching is performed on all edges incident to this vertex set. This branching strategy can be viewed as an aggregated version of branching on edges, thus intuitively causing a broader impact. It has been applied in some BPC (Fukushima et al. 2006, Jepsen et al. 2008, Pecin et al. 2017a, Yang et al. 2023) and B&C (Naddef and Rinaldi 2002, Baldacci et al. 2004, Lysgaard et al. 2004) algorithms. In a recent study, Silva and Uchoa (2022) proposed cluster branching, which can be viewed as a special case of branching on sets. It first divides the vertices into multiple clusters based on geographical location and then defines two sets of variables, that is, intra- and intercluster variables. Their empirical study demonstrated that this strategy could substantially reduce the CPU time for instances with clustered customer locations. Notably, it proved optimality for multiple CVRP and VRPTW open instances.

2.2. Branching Variable Selection

Once the type of variables to branch on is determined, the question reduces to how to make high-quality selections efficiently, which is the major focus of studies on branching strategies. The most straightforward approach is to select randomly without referring to any additional information, which, not surprisingly, leads to a large-sized BBT and thus undesirable ineffectiveness. A slightly more effective alternative is the

most fractional branching, which selects the variable with the largest fractionality (Augerat et al. 1995). The pseudocost branching, proposed by Bénichou et al. (1971) for solving general MIPs, has been widely adopted because of its high effectiveness and computational efficiency. It utilizes historical information to compute pseudocosts, which requires proper initialization. Applegate et al. (1995) introduced SB for solving the traveling salesman problem, which has been widely observed to produce small-sized BBTs. However, SB is generally not practical because of the time-consuming SB score computation. Linderöth and Savelsbergh (1999) combined the strengths of pseudocost branching and SB by using SB to initialize pseudocosts and improved the performance.

Pecin et al. (2017b) proposed to combine the most fractional branching, pseudocost branching, and SB into a 3PB in the context of solving VRPs by BPC algorithms. This approach comprises an initial screening phase and two estimation phases: one by solving LPs without CG and the other with heuristic CG. Such a phased strategy significantly reduces the number of branching candidates to consider, which, in turn, allows one to afford moderate extra computation while still selecting a branching variable of close-to-best quality with high chance. The 3PB, together with limited memory subset row cuts (Pecin et al. 2017c) and enumeration (Baldacci et al. 2008), helped to optimally solve open benchmark instances with up to 199 customers. The 3PB has since been adopted by the VRPSolver, further underscoring its efficacy and applicability.

2.3. ML for Solving VRPs

ML has proven effective in enhancing various existing algorithms for solving large-scale discrete optimization problems, including VRPs. The application of ML to approaching VRPs can be roughly categorized into two types. The first type seeks to directly construct heuristic solutions (Khalil et al. 2017, Nazari et al. 2018, Costa et al. 2020) or predict the feasibility of a given problem (Mai and Navet 2021, van der Hagen et al. 2024). However, these heuristic-based ML methods, especially graph neural network-based methods, struggle with generalization, scale poorly with instance size, and lack quality guarantees. Their performance degrades significantly as the number of nodes increases, both in terms of solution quality and inference time (Cappart et al. 2023). In contrast, our ML models are significantly less complex, and therefore, they demonstrate strong generalization capability while maintaining the optimality guarantee.

The second type focuses on accelerating some specific components in a sophisticated exact algorithm. Morabit et al. (2021) applied graph neural networks to pick columns likely to improve the objective value of the current RMP. Such selection significantly reduced

the size of the RMP, and thus, a 30% decrease in computing time for solving the root node was achieved. In another closely related work (Morabit et al. 2022), the authors applied supervised learning (logistic regression, random forest, and neural network) to identify arcs that are most likely to be used in an optimal solution. The unselected arcs were subsequently discarded, and the resulting residual graph was used for heuristic pricing. Readers can refer to Tahir et al. (2021) and Zhang et al. (2022) for additional works falling into this category.

Despite much success achieved in applying ML to enhance branching decisions for solving general MIPs (Khalil et al. 2016, Alvarez et al. 2017, Gasse et al. 2019, Yang et al. 2022), to the best of our knowledge, there is no known work on learning to branch in BPC algorithms for solving VRPs. A slightly related work is Pereira et al. (2022), where the authors used ML to assist in solving the crew pairing problem. Note that, instead of trying to solve it exactly, they aimed to compute a feasible solution via a branch-and-price heuristic. Building on the reviewed pioneering efforts, our paper seeks to fill this research gap.

3. Preliminaries

Before jumping into the intricacies of the proposed framework, to facilitate readability, we present necessary preliminaries about a general BPC method for solving VRPs. We first review the SPF with a focus on the associated dynamics in the solution process. Subsequently, we discuss the branching component, underscoring the differences between its usage in BPC and in B&C for solving MIPs. Lastly, we elaborate on an effective branching strategy widely adopted in BPC algorithms, which addresses the high computational overhead of SB while keeping the BBT size relatively small.

3.1. SPF

The standard SPF for VRPs, including the classic CVRP and VRPTW, takes the following general form:

$$\begin{aligned} \min \quad & \sum_{r \in \Omega} c_r x_r \\ \text{s.t.} \quad & \sum_{r \in \Omega} a_{ir} x_r = 1, \quad \forall i \in N, \\ & \sum_{r \in \Omega} w_{jr} x_r \leq w_j, \quad \forall j \in Q, \\ & x_r \in \{0, 1\}, \quad \forall r \in \Omega, \end{aligned} \quad (1)$$

where N , Q , and Ω denote the sets of customers to serve, the index set for side constraints, and the set of vehicle routes, respectively. The coefficient c_r represents the cost of route r . Coefficient a_{ir} is a binary indicator, with $a_{ir} = 1$ if route r visits customer i , and 0

otherwise. The binary decision variable x_r equals one if route r is selected in the solution and zero otherwise. It is important to note that it suffices to include only elementary routes in the SPF, that is, the customers of the route can only be visited exactly once. However, to expedite the CG process, *ng*-route relaxation (Baldacci et al. 2011) is often applied, which allows nonelementary routes. In this case, a_{ir} extends to a non-negative integer representing the number of times route r visits customer i . Inequalities $\sum_{r \in \Omega} w_{jr} x_r \leq w_j$, $\forall j \in Q$, represent the additional side constraints. For example, in the multidepot VRP (Contardo and Martinielli 2014), constraints $\sum_{r \in \Omega_j} x_r \leq m_j$, $\forall j \in D$, enforce the restriction on the fleet size, where D is the set of depots, m_j is the fleet size of depot j , and $\Omega_j \subseteq \Omega$ denotes the set of routes that start and end at depot $j \in D$.

Initially, only a small subset of routes $\Omega' \subseteq \Omega$ are included. Formulation (1) with Ω replaced by $\Omega' \cup \Omega''$ is referred to as the RMP, where Ω'' is the set of variables generated via CG. To solve the LP relaxation of the RMP, columns (variables) with negative reduced costs are generated iteratively via CG, also known as pricing. If the optimal solution to the RMP is fractional, branching is applied to help achieve integrality. We remark that the new variables complicate branching selection because historical data on their behaviors is lacking. Additionally, cutting planes increase the dimension of the dual problem and may even alter the structure of the pricing subproblem (Pecin et al. 2017b). Branching further complicates the situation by introducing branching constraints that accumulate as the tree grows deeper. Essentially, the BPC solution process involves alternating rounds of column and row generation, resulting in highly dynamic changes (Figure 1) to the RMP, which challenges existing learning-to-branch paradigms.

3.2. Branching Component

Although branching on a nominal variable in MIPs is a popular choice, it is not the case in the BPC context because imposing $x_r = 0$ is not straightforward for CG and is often ineffective as it creates extremely unbalanced BBTs. In contrast, branching on an edge in the underlying graph requires no change in CG and leads to a more balanced tree, making it a mainstream choice in BPC algorithms (Costa et al. 2019). Branching on a given edge can be generally achieved by adding a dense constraint of the form $\sum_{r \in \Omega} b_r x_r \leq 0$ or $\sum_{r \in \Omega} b_r x_r \geq 1$, where b_r counts the number of times route r traverses the edge. It is worth mentioning that such branching constraints also enforce restrictions on variables that have yet to be generated.

SB considers a set of branching candidates and performs tests by branching on each of them separately. Let z denote the optimal value of the parent LP. For a given candidate, let z_l and z_r be the LP values at the left and right child BBNs, respectively, when branched on. Then the SB score s of this particular branching choice is computed by the product rule (Achterberg 2007): $s = \max\{z_l - z, \epsilon\} \cdot \max\{z_r - z, \epsilon\}$, where $\epsilon > 0$ is typically set to 10^{-6} . SB then selects to branch on the one with the largest score. The calculation of pseudocost is similar to that of s by using historical information. Let $(\hat{x}_r)_{r \in \Omega' \cup \Omega''}$ be the solution to the parent LP. For a given edge e , let $\hat{x}(e)$ be the total flow on e computed by $\hat{x}(e) = \sum_{r \in \Omega' \cup \Omega''} a_{ir} \hat{x}_r$. Then the edge e can be considered as a branching candidate if $\hat{x}(e)$ is fractional. Let $\overline{\Delta z_l}$ and $\overline{\Delta z_r}$ denote the average value of $(z_l - z)$ and $(z_r - z)$ collected from historical branching, respectively. The pseudocost is computed as $\max\{(\hat{x}(e) - \lfloor \hat{x}(e) \rfloor) \cdot \overline{\Delta z_l}, \epsilon\} \cdot \max\{(\lceil \hat{x}(e) \rceil - \hat{x}(e)) \cdot \overline{\Delta z_r}, \epsilon\}$. In this study, we adopt this way for our pseudocost calculation.

However, calculating such z_l and z_r values for each candidate can still need a large number of CG

Figure 1. (Color online) Dynamic Changes of the RMP in the BPC Solution Process

$$\begin{aligned}
 & \text{min } \sum_{r \in \Omega'} c_r x_r + \sum_{r \in \Omega''} c_r x_r \\
 & \text{s.t. } \sum_{r \in \Omega'} a_{ir} x_r + \sum_{r \in \Omega''} a_{ir} x_r = 1, \quad \forall i \in N, \\
 & \quad \sum_{r \in \Omega'} w_{jr} x_r + \sum_{r \in \Omega''} w_{jr} x_r \leq w_j, \quad \forall j \in Q, \\
 & \quad \sum_{r \in \Omega'} s_{kr} x_r + \sum_{r \in \Omega''} s_{kr} x_r \leq s_k, \quad \forall k \in S, \\
 & \quad \sum_{r \in \Omega'} b_{lr} x_r + \sum_{r \in \Omega''} b_{lr} x_r \leq b_l, \quad \forall l \in B, \\
 & \quad x_r \in \{0, 1\}, \quad \forall r \in \Omega' \cup \Omega''.
 \end{aligned}$$

Initial variables and Generated variables are indicated by blue arrows pointing to the respective terms in the objective and constraints. Cutting planes and Branching constraints are indicated by blue arrows pointing to the respective constraints.

iterations, making this process time-consuming. To make SB faster, one can limit the number of candidates to test and apply heuristic CG in place of the much slower exact CG to solve the starting child LPs or even bypass CG completely. Without exact CG, the SB score computed is not precise, potentially compromising the quality of the branching decision. For the clarity of presentation, we thereafter refer to the procedure of branching on each candidate and solving the starting child LPs without any CG to compute an SB score as *LP testing* and refer to this SB score as *LP score*. The counterparts are called *exact testing* and *heuristic testing* when solving with exact CG and heuristic CG, respectively. The resultant SB scores are accordingly called *heuristic score* and *exact score*. Moreover, in this paper, unless stated otherwise, the term pseudocost always refers to the pseudocost computed using historical LP scores. Pseudocosts derived from heuristic and exact scores are termed *heuristic pseudocost* and *exact pseudocost*, respectively.

3.3. Three Phase Branching Selection

The 3PB is considered the state-of-the-art branching strategy in BPC algorithms for solving VRPs, which combines the aforementioned two ideas for enhancing SB. In particular, it runs progressively time-consuming testing on a gradually reduced candidate list and totally avoids performing exact CG. Basically, it balances the tradeoffs between the time consumed for branching decision making and the decision quality. As suggested by its name, the 3PB (Figure 2) comprises three phases—initial screening, LP testing, and heuristic testing—which gradually shorten the candidate list until a final decision is reached.

In the initial screening, the complete set of branching candidates, denoted by $\mathcal{C}$, is first divided into two groups: one with initialized pseudocosts and the other without. For the former group, the top $\hat{\theta}_p$ candidates, denoted by $\hat{\mathcal{C}}_p$, are selected based on pseudocosts, whereas for the other group, the top $\hat{\theta}_f$ candidates with the highest fractionality, denoted by $\hat{\mathcal{C}}_f$, are chosen. The parameters $\hat{\theta}_p$ and $\hat{\theta}_f$ are typically set equal and let $\hat{\theta}_{pf} := \hat{\theta}_p + \hat{\theta}_f$. The $\hat{\theta}_{pf}$ candidates in $\hat{\mathcal{C}}_p \cup \hat{\mathcal{C}}_f$ then go through LP testing and the top $\tilde{\theta}$ candidates ranked by LP scores constitutes the set $\tilde{\mathcal{C}}$. Lastly, heuristic testing is performed on candidates in $\tilde{\mathcal{C}}$, and the one with the largest heuristic score is the final decision.

4. Learning-to-Branch Framework

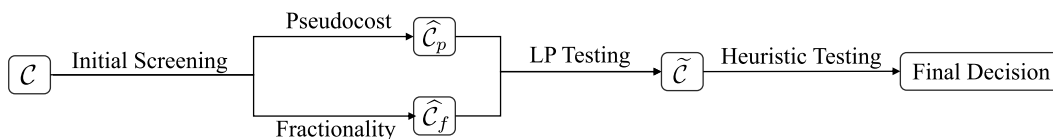
Similar to Khalil et al. (2016), Alvarez et al. (2017), and Yang et al. (2022), and the various related studies cited therein, our goal is to apply ML to mimic SB in a computationally efficient manner. Unfortunately, learning-to-branch paradigms used in Yang et al. (2022) that have been proven to work well in the context of solving MIPs fail to achieve any prediction accuracy above 20% according to our preliminary experiments, confirming that learning to branch in BPC algorithms is much more difficult. Such difficulty stems from causes explained in Section 1.1, which are addressed by our novel learning-to-branch framework.

4.1. High-Level Idea

As explained in Section 1.1, 3PB lacks the critical information necessary for efficiently making good branching decisions, and the three mentioned challenges prevent us from leveraging proven useful features from the literature for learning to optimize MIPs. In this section, we first identify new features to overcome the second and third challenges—the absence of historical information about recently generated (nominal) variables and the ineffectiveness of standard static features caused by the accumulated changes in the RMP. For the second challenge, we propose substituting the missing information with features of implicit variables. Because branching occurs on an edge in the graph—corresponding to a static variable in an equivalent compact formulation that remains unaffected by the CG process—historical data on these implicit variables can be collected throughout the solution process. To address the third challenge, our framework uses features derived from the graph (e.g., the depot and customer locations and the induced proximities), which offer consistent predictive power and require one-time computation, as a substitute for static RMP-derived features.

The first challenge—the unknown beforehand deviation of an ending child LP from its parent LP—is particularly difficult to overcome. Even with the features mentioned above, achieving accurate predictions for the deviation remains challenging. Performing exact CG, as is done in exact testing is the only reliable way to obtain this uncertain deviation. However, exact testing is often time-consuming, even when evaluating just two candidates. Our second stage aims to limit its use to necessary cases. Specifically, when the

Figure 2. Workflow of the 3PB



deviation between an ending child LP and its starting LP is expected to be minimal, exact testing for that starting child LP can be skipped without risk. However, directly predicting this deviation is as complex as predicting the exact SB score since knowing the deviation would allow for deriving the SB score. Drawing from the success of *ranking* methods in learning-to-branch tasks for MIPs, our second-stage model focuses on predicting the rank of the *deviation ratio*, ρ , for each starting child LP. Let $s := z_1 - z$ and $\hat{s} := z_2 - z$, where z , z_1 , and z_2 are the optimal values of the corresponding parent, starting, and ending child LPs, respectively. This deviation ratio, defined as $\rho := \frac{\hat{s}}{s} \geq 1$, reflects the similarity between the starting and ending child LPs and thus serves as a measure of the necessity for exact testing. By ranking the ρ values for all candidates—each with two starting child LPs (one from the left branch and the other from the right)—we can selectively limit exact testing to only those starting child LPs with high predicted deviation.

4.2. Workflow

Our framework builds on the cutting-edge handcrafted 3PB detailed in Section 3.3. Recall that 3PB has two primary weaknesses. The first is the less effective initial screening, which leads to solving a large number of (often in the hundreds) starting child LPs without CG in the LP testing phase. The second is that it needs to perform several heuristic testing to determine the best candidate from those identified by LP testing. Although the number of heuristic testing may not be very large, each testing involves multiple iterations of heuristic CG and solving intermediate LPs, making it potentially over ten times slower than an LP testing.

To overcome these two weaknesses, our framework employs two ML models: one before the LP testing phase (*M1*) and one after LP testing (*M2*), as illustrated in Figure 3. *M1* further selects promising candidates from the set given by initial screening, thereby reducing the need to solve hundreds of LPs without CG in the LP testing phase. *M2* aids in selecting a subset of starting child LPs for testing while ensuring

decisions match those from testing *all* starting child LPs. We term such testing as *partial testing*.

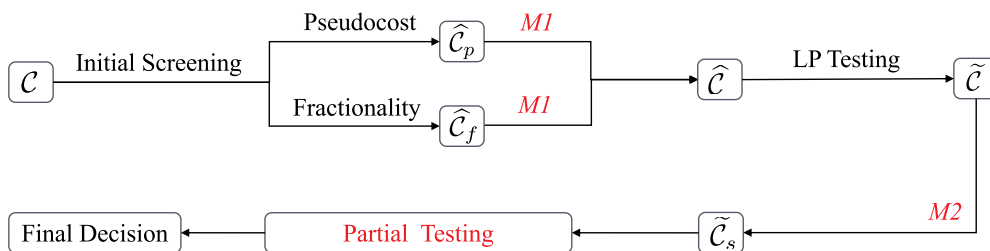
Specifically, in the first stage, *M1* mimics the behavior of performing exact testing on candidates that have passed the initial screening, with the learning goal of predicting the ones with the highest exact scores. In the second stage, *M2* predicts whether the starting child LP value (z_1) accurately approximates the ending child LP value (z_2) on the candidates that have passed LP testing. Generally speaking, starting child LPs expected to have a large deviation ratio will be subjected to partial testing for a more precise estimate of z_2 . Details on the second stage are provided in Section 4.4.

4.3. First-Stage Selection

We propose a phased design with two stages of ML (Figure 3) that define two sequential learning tasks. The first task utilizes easily computable features to filter out a large proportion of unpromising candidates from $\hat{C}_p$ and $\hat{C}_f$, producing a refined candidate set, $\hat{C}$. Notably, *M1* is applied separately to $\hat{C}_p$ and $\hat{C}_f$ to obtain $\hat{C}$, ensuring that some candidates from $\hat{C}_f$, if any, are retained. Without this treatment, our experiments indicate that *M1* tends to choose candidates almost exclusively from $\hat{C}_p$ when $\hat{C}_p \cup \hat{C}_f$ altogether is fed into *M1*, leading to unsatisfactory performance.

The reason is mainly twofold. First, the candidates in $\hat{C}_p$, that is, those with pseudocosts initialized, are of higher quality, in an average sense, than those in $\hat{C}_f$ because the pseudocost of a variable is initialized when it has successfully passed the initial screening. In contrast, the pseudocosts of candidates in $\hat{C}_f$ have not been initialized thus far, indicating that they failed such screening. Second, the pseudocosts get initialized fast as the BBT grows, causing $\hat{C}_f$ to vanish quickly. As a result, $\hat{C}_p$ constitute a vast majority of the training data, so *M1* naturally prioritizes members in $\hat{C}_p$. Nonetheless, some candidates from $\hat{C}_f$ can still be good branching choices, especially for branching decisions near the root of the BBT, where most of the pseudocosts have not yet been initialized. It is essential not to miss such candidates considering the decisions close to the root are generally of great importance (Pecin et al. 2017b, Pessoa et al. 2020).

Figure 3. (Color online) Workflow of Our Learning-to-Branch Framework



4.4. Second-Stage Selection

The key idea of the second-stage selection is to utilize the predicted ranking of ρ (output from M2) to avoid unnecessary testing. More specifically, this selection process relies on Conditions (2)–(4), proven to be sufficient in Proposition 1. It is based on the straightforward intuition: for two candidates $e_i, e_j \in \tilde{\mathcal{C}}$, if the current testing score, not necessarily the exact score, of e_i exceeds that of e_j , and the deviation ratio of e_i is no larger than that of e_j , then e_j can be removed from $\tilde{\mathcal{C}}$, as its exact score cannot surpass that of e_i . Further details are provided in Section 4.4.2.

It is rare, however, that these three conditions are satisfied for all candidate pairs (e_*, e_j) at the beginning, where $e_* \in \tilde{\mathcal{C}}$ has the highest exact score. To address this, Algorithm 1 conducts exact testing on a subset of the starting child LPs in a dynamic manner to ensure these conditions hold across all pairs (e_*, e_j) . Consequently, all such e_j will be removed from $\tilde{\mathcal{C}}$, leaving only e_* , thus achieving the selection goal.

4.4.1. Assumptions on the Behavior of M2. Let l_i and r_i denote the left and right starting child LPs, respectively, generated by adding the branching constraint associated with the i th candidate $e_i \in \tilde{\mathcal{C}}$ and define $\tilde{\mathcal{C}}_s := \bigcup_{i=1}^{|\tilde{\mathcal{C}}|} \{l_i, r_i\}$ as the set of all starting child LPs. Given the input features, M2 outputs a rank by the ρ values for each starting child LP in $\tilde{\mathcal{C}}_s$.

We formalize some assumptions based on the observed behavior of M2, which relate the deviation ratio, ρ , to the output ranking of M2, denoted by d .

Assumption 1. d is a function of the deviation ratio, that is, $d = h(\rho)$, where $h : [1, \infty) \rightarrow \mathbb{Z}_+ \cup \{-1\}$.

Assumption 2. h is a monotonically increasing function such that $h(\rho) = -1$ if and only if $\rho = 1$.

Assumption 3. $h(\rho_1) = h(\rho_2) \leq \underline{d}$ implies that $\max\{\frac{\rho_1}{\rho_2}, \frac{\rho_2}{\rho_1}\} \leq \delta$, where $\underline{d}$ and δ are predefined parameters.

If M2 achieves perfect accuracy, the assumptions can be justified as follows: (1) ρ should be the sole factor influencing the prediction d , thereby ensuring the existence of h ; (2) d should increase monotonically as deviation ratio ρ increases, aligning with the learning target of M2; and (3) based on our observation, a small d indicates that s is a strong approximation of $\hat{s}$, and thus the corresponding ρ_1 and ρ_2 should be close to one when $h(\rho_1) = h(\rho_2) \leq \underline{d}$. Although the same d value does not guarantee identical ρ values, a minor difference is expected because both ρ_1 and ρ_2 are close to one in this case. Therefore, it is reasonable to assume that $\max\{\frac{\rho_1}{\rho_2}, \frac{\rho_2}{\rho_1}\}$ is bounded by a constant δ .

4.4.2. Conditions to Ensure a Larger Exact Score. The conditions introduced here are fundamental to the

proposed partial testing procedure. They establish sufficient criteria to prove that e_i has a larger exact score than e_j , even without knowing their exact scores. To proceed, we first clarify the notation. Let S_k denote the difference between the optimal value of the *current* child LP $p_k \in \tilde{\mathcal{C}}_s$ and its parent LP value, and D_k represents the *current* deviation ratio. Note that current signifies dynamic determination based on the state of testing. If a child LP $p_k \in \tilde{\mathcal{C}}_s$ has not yet been tested, S_k is set to s_k , and D_k is set to d_k . Otherwise, S_k is updated to $\hat{s}_k$, and D_k is updated to $h(1) = -1$ to indicate that p_k has been tested. For clarity, when no superscript is attached, we do not distinguish the left or right child LP. When necessary, we employ the superscript l for the left child and r for the right; for example, S_i^l and S_i^r denote the S values of the corresponding left and right child LPs associated with branching candidate e_i . The current score of e_i is represented as $S_i^{l \times r} := S_i^l \times S_i^r$, and its exact score is denoted by $\hat{s}_i^{l \times r} := \hat{s}_i^l \times \hat{s}_i^r$.

Definition. We define e_i as a *better* branching candidate than e_j if the following conditions hold for the candidate pair (e_i, e_j) :

$$S_i^{l \times r} > S_j^{l \times r} \cdot \delta^{\phi_1 + \phi_2}, \quad (2)$$

$$(D_i^{\max} < D_j^{\max} \text{ and } \phi_1 = 0) \text{ or} \\ (D_i^{\max} = D_j^{\max} \leq \underline{d} \text{ and } \phi_1 = \mathbb{1}_{D_i^{\max} \neq -1}), \quad (3)$$

$$(D_i^{\min} < D_j^{\min} \text{ and } \phi_2 = 0) \text{ or} \\ (D_i^{\min} = D_j^{\min} \leq \underline{d} \text{ and } \phi_2 = \mathbb{1}_{D_i^{\min} \neq -1}), \quad (4)$$

where $D_i^{\max} := \max\{D_i^l, D_i^r\}$, $D_i^{\min} := \min\{D_i^l, D_i^r\}$, $D_j^{\max} := \max\{D_j^l, D_j^r\}$, and $D_j^{\min} := \min\{D_j^l, D_j^r\}$.

Condition (2) specifies that the score $S_i^{l \times r}$ for e_i must be greater than the score for e_j , scaled by a factor $\delta^{\phi_1 + \phi_2}$. Conditions (3) and (4) further require that the $D_i^{\max}$ and $D_i^{\min}$ values of e_i are either strictly less than those of e_j or, if equal, must not exceed the predefined threshold $\underline{d}$. Together, these conditions ensure that e_i can be regarded as a better branching candidate than e_j , because they provide sufficient evidence that e_i achieves a larger exact score than e_j , as demonstrated in Proposition 1.

Proposition 1. Conditions (2), (3), and (4) are sufficient to prove that e_i has a larger exact score than e_j ; that is, $\hat{s}_i^{l \times r} > \hat{s}_j^{l \times r}$.

By verifying the conditions outlined in Proposition 1, it is possible to replicate the selection made by SB without requiring explicit computation of the exact scores, thereby significantly reducing computational complexity. Given a pair of candidates, the one with a larger current score $S^{l \times r}$ is referred to as the *Leader*, and the other is the *Competitor*. The candidate $e_* \in \tilde{\mathcal{C}}$ that is

better than the others is called the *Winner*, and the goal of our selection process is to identify it with the minimum necessary testing.

4.4.3. Partial Testing Procedure. In the ideal scenario, these conditions would hold for all candidate pairs (e_i, e_j) from the very beginning, making e_* the winner, thereby eliminating the need for any additional testing. However, as indicated by our preliminary experiments, this rarely occurs in practice. Candidates with larger LP scores often tend to have larger ρ values, preventing the three conditions from holding simultaneously. To eventually satisfy these conditions, exact testing on some starting child LPs needs to be performed to update the $S^{l \times r}$ and D values.

The challenge lies in strategically selecting a subset of $\tilde{C}_s$ for testing to meet the conditions as efficiently as possible. For a given pair (e_i, e_j) with $S_i^{l \times r} > S_j^{l \times r}$, the strategy is to prioritize e_i , and perform testing on its starting LPs, that is, l_i and r_i . If, after testing, the updated $S_i^{l \times r}$ remains larger, then the conditions are satisfied, eliminating the testing need for e_j . On the other hand, starting with l_j (or r_j) may lower both $S_j^{l \times r}$ and D_j^l (or D_j^r), and testing for e_i is still required.

Algorithm 1 outlines the aforementioned partial testing procedure. The procedure begins by initializing S_k and D_k to s_k and d_k , respectively, for all starting child LPs $p_k \in \tilde{C}_s$. In the subsequent sorting step, the candidates are sorted based on their current scores $S^{l \times r}$. The top two candidates, e_i and e_j , are selected to check if they satisfy Conditions (2), (3), and (4). If these conditions are satisfied, e_j will be removed from $\tilde{C}$. If not, exact CG will be performed on p_k , which is chosen as either the left starting child LP l_i or the right r_i depending on the relative values of D_i^l and D_i^r . Following this, S_k will be updated to $\hat{s}_k$, and D_k will be updated to -1 , after which the sorting step will be repeated.

Algorithm 1 (Partial Testing Procedure)

Step 1. Sorting: If $|\tilde{C}| = 1$, **Stop**.

Sort candidates in $\tilde{C}$ in descending order by $S^{l \times r}$, identify top candidates e_i, e_j .

Step 2. Checking: Set $p_k \leftarrow \text{NULL}$.

If Condition (3) failed: if $D_i^l \geq D_i^r$, set $p_k \leftarrow l_i$, otherwise set $p_k \leftarrow r_i$.

If Condition (4) failed: if $D_i^l \leq D_i^r$, set $p_k \leftarrow l_i$, otherwise set $p_k \leftarrow r_i$.

Step 3. Testing:

If p_k equals l_i : Compute $\hat{s}_i^l$, and update $S_i^l \leftarrow \hat{s}_i^l$, $D_i^l \leftarrow -1$, $S_i^{l \times r} \leftarrow S_i^l \times S_i^r$;

If p_k equals r_i : Compute $\hat{s}_i^r$, and update $S_i^r \leftarrow \hat{s}_i^r$, $D_i^r \leftarrow -1$, $S_i^{l \times r} \leftarrow S_i^l \times S_i^r$;

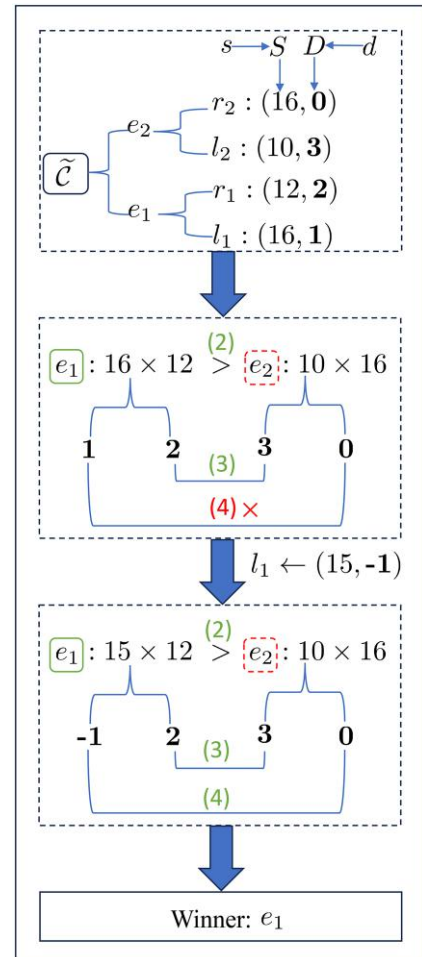
Otherwise: Eliminate e_j from $\tilde{C}$.

Go to Step 1.

An illustration example is provided in Figure 4, where $d = 3$, and the set of candidates $\tilde{C} = \{e_1, e_2\}$, with initial values $(S_1^l, D_1^l) = (16, 1)$, $(S_1^r, D_1^r) = (12, 2)$, $(S_2^l, D_2^l) = (10, 3)$, and $(S_2^r, D_2^r) = (16, 0)$. In Figure 4, the square around e_1 indicates that e_1 is the leader, and the dash square around e_2 signifies that e_2 is the competitor. Conditions (2), (3), and (4), are indexed as (2), (3), and (4), respectively, with the green color indicating a satisfied status and red for an unsatisfied status.

During the sorting step, e_1 is determined to be the leader because $S_1^{l \times r} = 16 \times 12 = 192$ is larger than $S_2^{l \times r} = 10 \times 16 = 160$. Consequently, e_2 is designated as the competitor. The algorithm proceeds to the checking step (Step 2), where Conditions (3) and (4) are evaluated. Because Condition (4) is not satisfied, l_1 is chosen to compute $\hat{s}_1^l$, which updates S_1^l from 16 to 15 and sets D_1^l to -1 . The algorithm then returns to the sorting step (Step 1). Despite the update, e_1 remains the leader because 15×12 is still greater than 10×16 . The checking step is repeated, and this time both Conditions (3) and (4) are satisfied, so no further testing is required,

Figure 4. (Color online) Example of the Partial Testing Process in the Proposed Framework



and e_2 is removed from $\tilde{C}$. This leaves e_1 as the only candidate in $\tilde{C}$ and thus the winner. Proposition 2 confirms that, after a finite number of iterations, the winner can be identified by Algorithm 1.

Proposition 2. *Algorithm 1 terminates in a finite number of steps and identifies the winner e_* that is better than any other candidate $e_j \in \tilde{C}$.*

4.4.4. Acceleration Technique. When $|\tilde{C}|$ is large, Algorithm 1 can still require significant computation to identify e_* . We propose to address it by reusing information obtained from testing that has already been conducted. Given Assumptions 2 and 3, Proposition 3 estimates the exact score by approximating ρ based on other known deviation ratio values.

Proposition 3. *Given a set of pairs $S := \{(\rho_\ell, h(\rho_\ell))\}_\ell \cup \{(M, M), (1, -1)\}$ with M sufficiently large and a candidate pair (e_i, e_j) . If $D_i = -1$, set $\rho_1 = 1$, otherwise, let $\rho_1 = \min_{(\rho_k, h(\rho_k)) \in S} \{\rho_k : h(\rho_k) > D_i\}$. Define $\rho_2 := \max_{(\rho_k, h(\rho_k)) \in S} \{\rho_k : h(\rho_k) < D_j\}$. It holds that e_i has a larger exact score than e_j , that is, $\hat{s}_i^{l \times r} > \hat{s}_j^{l \times r}$, if*

$$S_i^{l \times r} > S_j^{l \times r} \cdot \delta_a \quad (5)$$

is satisfied, where $\delta_a = \min\{a(D_i^l, D_j^l) \cdot a(D_i^r, D_j^r), a(D_i^l, D_j^r) \cdot a(D_i^r, D_j^l)\}$, and

$$a(D_1, D_2) = \begin{cases} \min\{\delta, \frac{\rho_1}{\rho_2}\}, & \text{if } D_1 = D_2 \leq \underline{d}, \\ \min\{1, \frac{\rho_1}{\rho_2}\}, & \text{if } D_1 < D_2, \\ \frac{\rho_1}{\rho_2}, & \text{if } D_1 > D_2 \text{ or } D_1 = D_2 > \underline{d}. \end{cases}$$

Condition (5) is checked at the beginning of Step 2 in Algorithm 1. If it is satisfied, Step 2 terminates directly with p_k set to NULL and it moves directly to Step 3. In practice, testing four starting child LPs is often sufficient for Condition (5) to conclude that no further testing is necessary. This significantly accelerates the entire selection process, especially when $|\tilde{C}|$ is 10 or larger.

Remarks. Although the heuristic testing in 3PB and the proposed partial testing procedure in our 2LBB expedite the selection through different mechanisms, the best numerical performance is achieved when these approaches are combined. Specifically, we replace the exact CG with heuristic CG in Algorithm 1 in our implementation when performing the testing. This is motivated by the observation that heuristic CG is significantly faster than exact CG, and more importantly, the optimal value obtained from heuristic CG serves as a reliable way to approximate that of the ending child LP, leading to the same selection outcome as exact CG in most cases. Although Propositions 1 and 3 may no

longer guarantee sufficient conditions for certifying a larger exact score, the reliability of these approximations ensures that such cases are rare in practice.

4.5. Learning Methodology

The two ML models, $M1$ and $M2$, in our ML module aim to predict the correct ranking with different learning goals. For $M1$, the goal is to rank candidates based on their exact scores, whereas for $M2$, the goal is to rank candidates based on the deviation ratio. *Learning to rank*, compared with more common supervised learning methods such as regression or classification, tends to achieve higher accuracy in our task because the learning objective is relatively easier.

The input for a ranking task typically consists of multiple lists, each containing several items. For the k th list with n_k items, let $\vec{f}_i^k \in \mathbb{R}^d$ denote the d -dimensional feature vector of item i , and $r_i^k \in \{1, \dots, n_k\}$ denote its rank in descending order based on exact scores or the deviation ratio. Each list can be expressed as (x^k, y^k) , where $x^k = \{\vec{f}_1^k, \dots, \vec{f}_{n_k}^k\}$ and $y^k = \{r_1^k, \dots, r_{n_k}^k\}$, forming training data for learning-to-branch tasks. However, directly using ranks as labels is unnecessary because candidates with similar scores typically have comparable performance. To reduce overfitting and enhance generalization, we recompute labels $\tilde{y}_i^k$ following the approach in Yang et al. (2022):

$$\tilde{y}_i^k = W - \max\left\{0, \left\lceil \frac{\tilde{s}^k - s_i^k}{\tilde{s}^k - s_{(n_k)}^k} \cdot W \right\rceil\right\}, \quad \text{where} \\ \tilde{s}^k = \alpha \cdot s_{(1)}^k, \quad \alpha \in (0, 1). \quad (6)$$

Candidates with scores above the threshold $\tilde{s}^k$ (commonly set with $\alpha = 0.8$) receive the maximum label W , indicating equally optimal choices for branching.

We characterize each branching candidate using three types of features: $\mathcal{F}_E$, $\mathcal{F}_I$, and $\mathcal{F}_D$. The $\mathcal{F}_E$ features provide detailed LP-related information, including dual solution values and changes in LP objective values obtained from solving child LPs without using CG, which are computationally expensive. To the best of our knowledge, the $\mathcal{F}_E$ features are completely new in the literature. In contrast, $\mathcal{F}_I$ and $\mathcal{F}_D$ are inexpensive to compute. The $\mathcal{F}_I$ features capture the fundamental properties of candidates that are independent of previous LP results, whereas the $\mathcal{F}_D$ features are derived from previously computed $\mathcal{F}_E$ values. Together, these features support a two-stage learning approach ($M1$ and $M2$), which effectively selects and ranks branching candidates. For detailed descriptions of feature definitions, computations, data collection, and training procedures, please refer to Section EC.3 of the e-companion.

5. Theoretical Model for Determining the Number of Candidates

Branching strategies, such as the SB, 3PB, and 2LBB, require performing computationally intensive testing on a set of branching candidates to achieve small-sized BBTs. Nonetheless, determining the size of the set to test so that the overall solution time is minimized remains challenging because of a lack of a systematic way to balance the tradeoff between the time consumed for making a single branching decision and the decision quality. The size is typically decided by a trial-and-error numerical approach and remains unchanged once decided. In Section EC.7.2 of the e-companion, we numerically decide a good static $\hat{\theta}$ for the 2LBB as a baseline. In this section, we propose a novel theoretical model that theoretically decides the best size based on the solution progress, which is the first to embed a distribution of the local branching improvements into a tree size estimation. More precisely, we derive the so-called *best K formula* (BKF) to dynamically adjust the size to achieve the minimum total computational time under our assumptions.

The BKF is broadly applicable to branching strategies that employ one-phase testing, such as SB, or multiphase testing where either only one phase's duration can vary (e.g., 3PB) or one phase dominates the overall computation time (e.g., 2LBB). We refer to such a phase as the *key testing phase*. In 3PB and 2LBB, LP testing generally serves as the key phase. Notably, in 2LBB, this phase often accounts for more than 80% of the total branching time. Therefore, BKF determines $\hat{\theta}_{pf}$ in 3PB and $\hat{\theta}$ in 2LBB. For multiphase strategies, the applicability of BKF hinges on the assumption that the total branching variable selection time is approximately equal to the time spent on the key testing phase, plus $\tilde{c}$ representing the combined duration of other phases. If $\tilde{c}$ is negligible compared with the key phase, it can be treated as a constant. Unless stated otherwise, all proofs related to these derivations are included in Section EC.2 of the e-companion.

5.1. Derivation of the BKF

Given a set of n branching candidates, we seek to decide the number, k , of candidates randomly chosen from this set to go through the key testing phase such that the minimum possible time for solving the problem is achieved. Our theoretical model builds on the following assumptions.

Assumption 4. *The BBT is a binary tree and thus $n_{\text{inner}} = n_{\text{leaf}} - 1$, where n_{inner} is the number of inner BBNs, and n_{leaf} is the number of leaf BBNs. The time for solving each inner BBN consists of those consumed by branching, pricing, and adding cuts. For leaf BBNs, branching is not needed and the time only comprises the other two parts.*

Assumption 5. *It takes a constant time $2t$ to perform testing for a single branching candidate (two branches), a constant time $\tilde{c}$ for other phases in making a branching decision, and a constant time c' for pricing and adding cuts at a BBN. Moreover, $c \geq t$, where $c := c' + \frac{\tilde{c}}{2}$.*

Assumption 6. *The tree size estimation follows Le Bodic and Nemhauser (2017). For a given branching candidate, the corresponding dual bound improvements at the left and right child BBNs, denoted by o_1 and o_2 , respectively, remain constant with $o_1 \geq o_2 \geq 1$. We define testing score r to be the geometric mean of o_1 and o_2 ; that is, $r := \sqrt{o_1 \cdot o_2}$. The value of r varies for different branching candidates.*

Assumption 7. *The testing score for the n branching candidates take distinct values from $\{\frac{1}{n} \cdot R : j = 1, \dots, n\}$, where R is maximum testing score. The value of r_i for candidate i is not known until it undergoes the testing.*

Let $N(G)$ denote the size of the subtree rooted at the current BBN with a local optimality gap of G . In particular, for each leaf BBN, $N(0) = 1$. According to Le Bodic and Nemhauser (2017), $N(G)$ can be approximated by $N(G) = \varphi^{G-F} N(F)$, where F is no larger than G , and φ is the unique solution to the equation $x^{o_1} - x^{o_1-o_2} - 1 = 0$. Given that φ does not admit a closed form, analyzing the properties of equations involving φ is complex. Therefore, we propose to approximate φ by $\varphi_g := 2^{\frac{1}{g}}$. Theorem 1 characterizes the quality of such approximation.

Theorem 1. *The approximation error $|\frac{\varphi_g - \varphi}{\varphi_g}| \leq \frac{2^{-\frac{1}{g_1 v}} |2^{\frac{1}{g}} - 2^{-v+\frac{1}{g}} - 1|}{o_1 v^2}$, where $v := \sqrt{\frac{o_2}{o_1}} \in (0, 1]$. Moreover, the error is monotonically decreasing in terms of v when $v \geq \frac{1}{3}$.*

Theorem 1 bounds the approximation error. In particular, when $o_1 = 10$ and $v = \sqrt{\frac{1}{2}}$, which is typical for VRP instances, the error is no greater than 0.006. It is worth mentioning that this approximation only needs to be applied to the best one from the set of candidates considered and a good candidate usually has a value of o_2 close to that of o_1 , making v close to one and thus the error bound close to zero. Consequently, φ_g serves as a good approximation, and the tree size $N(G)$ is estimated to be $N(F) \cdot 2^{\frac{G-F}{g}}$ in our subsequent analysis.

The total computation time for the subtree is calculated as $T(k) = (c' + \tilde{c} + 2kt)n_{\text{inner}} + c' \cdot n_{\text{leaf}} = c'N(G) + (\frac{\tilde{c}}{2} + kt)(N(G) + 1) \approx (c' + \frac{\tilde{c}}{2} + kt)N(G) = (c + kt)N(G)$. When the calculation is performed at the root, $T(k)$ estimates the total time it takes to solve the problem. Combining the above two equations leads to

$$T(k) \approx (c + kt)N(F) \cdot 2^{\frac{G-F}{g}}. \quad (7)$$

In BPC algorithms, enumeration tends to succeed when the local gap is small enough, helping close a BBN fast. Therefore, the subtree rooted at the current

BBN, in this case, approximately has a constant size. In other words, when F is close to zero, $N(F)$ can be considered constant that is independent of k or r . If $N(F)$ can be estimated accurately, such information should be exploited. Otherwise, we can always use the case $F = 0$ with $N(0) = 1$ in Equation (7).

Intuitively speaking, increasing the number of candidates to test is likely to result in a higher testing score because of the increased probability of identifying the best candidate. The monotone relationship between r and k poses challenges in determining the value of k that minimizes $T(k)$. To address this, we first build a quantified connection between r and k . Let $j_1, \dots, j_k$ be the k randomly selected from the n candidates. After testing, the candidate with the highest score among the k tested ones is selected. Thus, in view of Assumption 7, the final testing score $r = \max\{r_{j_1}, \dots, r_{j_k}\}$ is a random variable. We proceed by examining the following two cases.

(i) Pure random selection. This case applies when there is no prior knowledge about the n candidates such as in the SB. The expected value of $T(k)$ is computed by

$$\begin{aligned} \mathbb{E}[T(k)] &= \mathbb{E}\left[(c + kt)N(F) \cdot 2^{\frac{G-F}{r}}\right] \\ &= (c + kt)N(F) \sum_{i=k}^n \frac{\binom{i-1}{k-1}}{\binom{n}{k}} 2^{\frac{G-F}{i}R}. \end{aligned} \quad (8)$$

(ii) Conditional random selection. The selection is conditioned on that only candidates with a testing score larger than $\frac{(n-m)R}{n}$ will be selected:

$$\begin{aligned} T_k &:= \mathbb{E}[T(k)|m] = \mathbb{E}\left[(c + kt)N(F) \cdot 2^{\frac{G-F}{r}} | m\right] \\ &= (c + kt)N(F) \sum_{i=k}^m \frac{\binom{i-1}{k-1}}{\binom{m}{k}} 2^{\frac{G-F}{i+n-m}R}. \end{aligned} \quad (9)$$

Case (ii) applies to the usage of $M1$ because $M1$ possesses some basic prediction power enabling it to effectively exclude $(n - m)$ unpromising ones. The output $k = \hat{\theta}$ candidates are considered as being randomly selected from the best m ones. When $m = n$, case (ii) reduces to (i), so case (i) can be considered as a special case. Therefore, we focus on case (ii) in the following discussion.

Proposition 4. *Given two branching strategies with a respective constant number of testing k_1 and k_2 ($k_1 > k_2$) and respective m values of m_1 and m_2 , the ratio of their expected solving time for the same instance $\tau := \frac{\mathbb{E}[T(k_1)|m_1]}{\mathbb{E}[T(k_2)|m_2]}$ decreases as ω increases, where $\omega := \frac{\epsilon}{t}$.*

The above proposition can be readily proved as follows. For the same instance, c and t remain unchanged across different strategies. In view of Equation (9), it

follows that τ is proportional to $\frac{\omega+k_1}{\omega+k_2}$. Because $k_1 > k_2$ and $\frac{\omega+k_1}{\omega+k_2} = 1 + \frac{k_1-k_2}{\omega+k_2}$, we have that τ decreases with ω . Proposition 4 suggests that for instances with a larger ω , overestimating the parameter k is less detrimental when $\tau > 1$. This insight can provide guidance on tuning k for instances with distinct structures. Additionally, upon successful enumeration at a BBN, pricing becomes much faster; that is, ω significantly decreases. In this case, a larger k is less desirable, and thus k should be reduced. Such a change in ω will be further studied numerically in Section 6.

Proposition 5. *Given two branching strategies selecting the same number of candidates, that is, $k_1 = k_2 = k$, but with $m_1 > m_2$, it follows that $\tau > 1$.*

Proposition 5 indicates that for two strategies using the same number of testing but differ in their m values, the one with a larger m will have a longer computational time than the other. Intuitively, a smaller m suggests a higher selection quality, leading to a more effective branching scheme. This correlation between m and τ is further supported by the comparison of the two case studies in Sections 6.1 and 6.2.

Let $k^* \in \arg \min_{1 \leq k \leq m} T_k$; that is, when k is set to k^* , the expected computational time is minimized. When m is relatively small, k^* can be obtained by computing T_k for $k = 1, \dots, m$, and select the smallest one. However, for large m , such computations become prohibitively expensive. In the next section, we provide an efficient way to compute k^* .

5.2. Asymptotic Study

In this section, we analyze the asymptotic behavior of the total computational time T_k as $n \rightarrow \infty$, which gives rise to a practical way of dynamically adjusting the key parameter $\hat{\theta}$ for $M1$. In this analysis, $M1$ is considered to have a fixed prediction power in the sense that its m value grows proportionally with n . In other words, $\alpha := \frac{m}{n}$ is a constant with $\alpha \in (0, 1)$. We demonstrate that k^* can be computed without explicitly computing T_k for every k in the range $\{1, \dots, m\}$. More specifically, we narrow down the possible values of k^* to just several options. Additionally, we establish that as $n \rightarrow \infty$, T_k possesses a definite integral representation. This integral form facilitates a more efficient computation of T_k . Let $\mathbb{E}[r|m]$ be the conditional expectation of r given m . Lemma 1 gives a closed form for computing it.

Lemma 1. *The conditional expectation of r can be computed by $\mathbb{E}[r|m] = \sum_k^m \frac{\binom{i-1}{k-1}}{\binom{m}{k}} \frac{i+n-m}{n} R = \frac{(n+1)k+n-m}{n(k+1)} R$.*

Define $\underline{T}_k := (c + kt)N(F)2^{\frac{G-F}{\mathbb{E}[r|m]}}$. In view of the convexity of the function $f(x) = 2^{1/x}$ and Jensen's inequality, $\underline{T}_k$ lower bounds T_k . For any given $k \in \{1, \dots, m\}$,

Theorem 2 establishes the existence of limits for T_k and $\underline{T}_k$ as $n \rightarrow \infty$, which enables us to focus on $\underline{T}_k$ when analyzing the asymptotic behavior of T_k .

Theorem 2. For $k \in \{1, \dots, m\}$, the limits $\hat{T}_k := \lim_{n \rightarrow \infty} T_k$ and $\hat{\underline{T}}_k := \lim_{n \rightarrow \infty} \underline{T}_k$ exist. Moreover, $\hat{T}_k = (c + kt)kN(F) \int_0^1 x^{k-1} B_{\alpha+1-\alpha} dx$ and $\hat{\underline{T}}_k = (c + kt)N(F)B_{1-\alpha/(k+1)}^{1-\alpha/(k+1)}$, where $B := 2^\beta$ and $\beta := \frac{G-F}{R}$.

Theorem 3. The sequence $\hat{U}_k := (c + kt)kN(F)B_{1-\alpha}^{\frac{1}{1-\alpha}} (\frac{\alpha}{1-\alpha} \ln(B))^{-k} \gamma(k, \frac{\alpha}{1-\alpha} \ln(B))$ upper bounds $\hat{T}_k$, where $\gamma(s, x) := \int_0^x t^{s-1} e^{-t} dt$ is the lower incomplete gamma function.

For any given $\hat{k} \in \{1, \dots, m\}$, we define $\Delta_{\hat{k}} := \{1 \leq k \leq m : \hat{\underline{T}}_k > \hat{U}_{\hat{k}}\}$. According to Theorems 2 and 3, $\hat{T}_k$ is lower bounded by $\hat{\underline{T}}_k$ and upper bounded by $\hat{U}_k$. Consequently, any $k \in \Delta_{\hat{k}}$ cannot achieve the smallest $\hat{T}_k$. Furthermore, the following Theorem 4 describes the monotonicity property of $\hat{\underline{T}}_k$ and suggests a convenient way to determine $\Delta_{\hat{k}}$ without computing $\hat{\underline{T}}_k$ for every $k \in \{1, \dots, m\}$.

Theorem 4. $\hat{\underline{T}}_k$ is monotonically decreasing for $k \in \{1, \dots, \underline{k}^*\}$ and monotonically increasing for $k \in \{\underline{k}^*, \dots, m\}$, where $\underline{k}^* = \max\{\arg \min_{1 \leq k \leq m} \hat{\underline{T}}_k\}$.

Combining the lower and upper bounds of $\hat{T}_k$ with its monotonicity, we can narrow down the range of k^* to a small number of options as shown in Theorem 5. In Proposition 6, we further provide an explicit expression for $\underline{k}^*$, enabling us to decide the value of k^* with even less computation.

Theorem 5. Let k'_l be the largest $k < \underline{k}^*$ and k'_r be the smallest $k > \underline{k}^*$, respectively, such that $\hat{\underline{T}}_k \geq \hat{U}_{\underline{k}^*}$. It then follows that k^* belongs to the set $\{k'_l + 1, \dots, k'_r - 1\}$.

In view of the monotonicity of $\hat{\underline{T}}_k$, both k'_l and k'_r can be efficiently computed.

Corollary 1. When $\hat{\underline{T}}_{\underline{k}^*} + 1 \geq \hat{U}_{\underline{k}^*}$ and $\hat{\underline{T}}_{\underline{k}^*} - 1 \geq \hat{U}_{\underline{k}^*}$, it holds that $k^* = \underline{k}^*$.

Corollary 1 provides a sufficient condition for $k^* = \underline{k}^*$, which is a special case of Theorem 5 with $k'_l = \underline{k}^* - 1$ and $k'_r = \underline{k}^* + 1$. In this case, there is no need to compute any $\hat{T}_k$. It is worth mentioning that the lower incomplete gamma function can be computed very efficiently (in microseconds) by many existing numerical libraries (e.g., Boost C++ library), rendering the evaluation of $\hat{U}_k$ significantly faster than that of $\hat{T}_k$.

Corollary 2. Let k'_l be the largest $k < \underline{k}^*$ and k'_r be the smallest $k > \underline{k}^*$, respectively, such that $\hat{\underline{T}}_k \geq \hat{\underline{T}}_{\underline{k}^*}$, it follows that $k'_l \geq k'_l$ and $k'_r \leq k'_r$. Moreover, $k^* \in \{k'_l + 1, \dots, k'_r - 1\}$.

Corollary 2 can be proved straightforwardly. According to Theorem 3, we have $\hat{\underline{T}}_{\underline{k}^*} \leq \hat{U}_{\underline{k}^*}$. In view of the

monotonicity of the $\hat{\underline{T}}_k$ and the definitions of k'_l , k'_r , k'_l , and k'_r , we have $k'_l \geq k'_l$ and $k'_r \leq k'_r$. Therefore, Corollary 2 further narrows the range of k^* . For $k \leq k'_l$, it follows that $\hat{T}_k \geq \hat{\underline{T}}_k \geq \hat{\underline{T}}_{k'_l} \geq \hat{\underline{T}}_{\underline{k}^*}$, which leads to $k^* \geq k'_l + 1$. For $k \geq k'_r$, we also have the same inequality, proving the other direction; that is, $k^* \leq k'_r - 1$. Therefore, k^* belongs to the set $\{k'_l + 1, \dots, k'_r - 1\}$. Let

$$k_1 = \sqrt{(\alpha \beta \ln \sqrt{2})(\alpha \beta \ln \sqrt{2} + 2\alpha + 2\omega - 2)} + \alpha \beta \ln \sqrt{2} + \alpha - 1. \quad (10)$$

Proposition 6. If $k_1 > 1$, then $\underline{k}^* \in \arg \min_{k \in \{\lfloor k_1 \rfloor, \lceil k_1 \rceil\}} \hat{\underline{T}}_k$.

Proposition 6 computes $\underline{k}^*$ by evaluating the value of k_1 and suggests that when n is sufficiently large, $\underline{k}^*$ tends to be a constant.

The procedure for efficiently determining the value of k^* when n is large is outlined as follows.

1. Compute k_1 by Equation (10). If $k_1 \leq 1$, then $\underline{k}^* = 1$, otherwise, pick $\underline{k}^* \in \arg \min_{k \in \{\lfloor k_1 \rfloor, \lceil k_1 \rceil\}} \hat{\underline{T}}_k$.
2. Check if $\hat{\underline{T}}_{\underline{k}^*} + 1 \geq \hat{U}_{\underline{k}^*}$ and $\hat{\underline{T}}_{\underline{k}^*} - 1 \geq \hat{U}_{\underline{k}^*}$ hold. If so, then set $k^* = \underline{k}^*$. Otherwise, compute k'_l , k'_r and pick $k^* \in \arg \min_{k \in \{k'_l + 1, \dots, k'_r - 1\}} \hat{\underline{T}}_k$.

6. Numerical Study

Our extensive numerical study is performed using RouteOpt, an exact solver for the CVRP and VRPTW, achieving state-of-the-art performance. Some implementation details of RouteOpt are included in Section EC.5 of the e-companion. Although the default configuration of RouteOpt, which activates all optimization techniques, is most suitable for practical use, the simultaneous deployment of cutting planes, arc elimination, and enumeration at all BBNs can significantly interfere with the branching process. To clearly demonstrate the effectiveness of our methodology, we also evaluate a second configuration where cutting planes and arc elimination are applied only at the root node, with enumeration completely disabled. Restricting cutting planes to the root node during branching studies is standard practice (Khalil et al. 2016, Alvarez et al. 2017, Yang et al. 2022), and methods such as arc elimination and enumeration are not typically used in standard B&C algorithms. For convenience, we denote experiments conducted with the default (first) and second configurations of RouteOpt as Settings I and II, respectively (Table 2). In Sections 6.1 and 6.2, we

Table 2. Summary of the Configurations Used in the Two Experimental Settings

Setting	Cutting plane	Arc elimination	Enumeration
I	All BBNs	All BBNs	Activated
II	Root node	Root node	Disabled

present two case studies on the CVRP and VRPTW, respectively, that demonstrate the effectiveness of the 2LBB derived from the proposed framework. Finally, Section 6.3 is dedicated to our attempt to solve open CVRP instances. Preliminary experiments performed to determine and justify the design of some key components of our learning-to-branch framework are presented in Section EC.7 of the e-companion.

For the instances tested in this study, we adopt a systematic naming convention to categorize and clarify the characteristics of each batch. The naming format is *problem-size-replication-type*, where *problem* specifies the type of problem (e.g., VRPTW or CVRP); *size* denotes the number of customers, with *V* indicating a varied size within the batch; *replication* represents the number of instances in the batch; and *type* identifies the category of instances, including *0*, *1*, *X*, and *H2*. Specifically, *0* and *1* correspond to newly generated instances, where *0* is used for training data generation and *1* for performance evaluation. The *X* category refers to the *X* series benchmarks for CVRP (Uchoa et al. 2017), whereas *H2* denotes type 2 instances for VRPTW (Homburger and Gehring 2005). We used instance batches CVRP-120-90-1, CVRP-150-90-1, CVRP-180-90-1, and CVRP-V-42-X for CVRP and VRPTW-160-60-1, VRPTW-180-60-1, VRPTW-200-60-1, and VRPTW-200-22-H2 for VRPTW to demonstrate the improvement achieved by the learned branching strategy when integrated into RouteOpt. Further details on the generation, attributes of these instances, and the rationale behind their selection can be found in Section EC.4 of the e-companion.

In our experiments, two sets of computing resources were used: HiPerGator and Dell workstations. HiPerGator is a supercomputing cluster at the University of Florida. We used the hpg-default partition running Red Hat Enterprise Linux 8.8, where each node is equipped with two AMD EPYC 770264-core processors @2.0 GHz and 1003 GB RAM. On Hipergator, the code was compiled by g++ 12.2.0. Our group has access to 1080 CPU cores and 8640 GB RAM, which allows us to run hundreds of jobs simultaneously. Training data generation was efficiently conducted using this computing resource. For experiments necessitating precise time comparisons, we used four identical Dell workstations. Each workstation runs on Ubuntu 20.04 LTS and is equipped with the 12th Gen Intel(R) Core(TM) i9-12900K CPU @3.2 GHz and 128 GB RAM. On the workstations, g++ version 9.4.0 was used to compile the code. For all experiments in this section, Gurobi 10.0.0 (Gurobi Optimization, LLC 2023) was used as the LP and MIP solver inside our RouteOpt, except in Section 6.1.4, where CPLEX 22.1.0 (IBM Corporation 2023) was employed to ensure a fair comparison with VRPSolver. All code was run using a single thread. The source code of RouteOpt, together with all the ML

code and CVRP and VRPTW instances, has been made publicly available on GitHub.

6.1. Case Study I: CVRP

In this case study, we use the CVRP to demonstrate the performance of our framework. We integrate the trained ML module into RouteOpt and evaluate the performance improvement by solving instances in batches CVRP-120-90-1, CVRP-150-90-1, CVRP-180-90-1, and CVRP-V-42-X, aiming to highlight the superiority of 2LBB over the benchmark 3PB and emphasize its generalization capability. Based on preliminary experiments in Section EC.7 of the e-companion, we set $\hat{\theta}_{pf} = 100$, $\hat{\theta} = 10$, and $\tilde{\theta} = 3$ for 2LBB, with the only difference for 3PB being $\tilde{\theta} = 2$. For the dynamic version, we assign $\eta = 0.72$, whereas α is set to 0.60 for 3PB-dy and 0.37 for 2LBB-dy, respectively. For the parameters of the second-stage selection, we set $\underline{d} = 2$ and $\delta = 1$. To assess the performance difference due solely to branching, experiments on batches CVRP-120-90-1, CVRP-150-90-1, and CVRP-180-90-1 are conducted under experimental Setting II, using the optimal value u^* or $u^0 := (1 + 0.1\%)u^*$ as the upper bound. To further demonstrate its generalization across different settings and compatibility with other techniques, CVRP-V-42-X is tested under Setting I. Details on training the ML models are included in Section EC.9.1 of the e-companion.

6.1.1. Superiority of the 2LBB. We compare the performance of RouteOpt using the 2LBB and the benchmark 3PB for instance batches CVRP-150-90-1. Additionally, we include two corresponding dynamic adjustments, that is, 3PB-dy and 2LBB-dy, which leverage the BKF to dynamically determine the number of time-consuming testing conducted. We report the time for solving the root node (Root/s), the root gap (Root gap/%), LP testing time (LP/s), heuristic testing time (Heuristic/s), the overall solution time (CPU/s), and the number of nodes explored (No. of nodes). Tables 3 and 4 present the geometric means of these metrics when two upper bounds (UBs) are used, that is, u^* and u^0 , respectively. Detailed results for each instance are included in Sections EC.10.1 and EC.10.2 of the e-companion.

We observe that when u^* is used as the UB, employing the learning models is beneficial, regardless of whether BKF is applied. Specifically, when BKF is not used, the time reduction (2LBB versus 3PB) can reach 36%, whereas with BKF, the time reduction (2LBB-dy versus 3PB-dy) is approximately 27%. This lower ratio suggests that BKF may play a more significant role in the 3PB strategy. Indeed, in lower-level branching decisions, where limited historical information is available, the performance of the initial screening—on which 3PB heavily relies—is significantly deteriorated. However,

Table 3. Comparison of 3PB, 2LBB, 3PB-dy, and 2LBB-dy When u^* Serves as the UB

Version	Root/s	Root gap/%	LP/s	Heuristic/s	CPU/s	No. of nodes
3PB	24.5	0.13	218.1	139.8	570.2	297.9
2LBB			166.2	29.2	362.5	222.1
3PB-dy			152.0	108.9	436.8	245.5
2LBB-dy			102.9	21.8	317.2	270.1

when incorporating $M1$, 2LBB is less affected by the lack of historical information. Consequently, adopting a strategy of testing more at lower levels and less at higher levels can be expected to enhance the performance of 3PB more effectively.

From the perspective of the node metric, we observe less node reduction (25%) than time reduction (36%) when comparing 2LBB with 3PB. The additional time reduction arises from the second stage, where fewer heuristic testing are required while maintaining a similar tree size, as shown in Section EC.7.3 of the e-companion. This time saving also causes 2LBB to require less time on average to solve a BBN (excluding LP testing), that is, a smaller c , which further contributes to different behavior observed for BKF. Specifically, 3PB-dy processes 18% fewer nodes than 3PB, whereas 2LBB-dy processes 22% more nodes than 2LBB. Note that in BKF, a smaller c often leads to fewer testing being performed and thus a larger tree size. Despite these differences in tree size, both 3PB and 2LBB benefit from BKF, achieving time reductions of 23% and 13%, respectively. Overall, the 2LBB enhanced with BKF achieves the best performance, with a 44% time reduction compared with the baseline 3PB.

For the case where u^0 is provided as the UB, similar to the conclusion drawn from Table 3, 2LBB still outperforms 3PB regardless of whether BKF is applied. However, the time reduction of 2LBB versus 3PB decreases to 31%. This reduction also applies to the versions using BKF. Further investigation reveals that, although 2LBB finds better solutions or identifies the optimal solution more quickly than 3PB, the ratio of the time spent finding the solution to the overall solution time is lower for 3PB. Specifically, as shown in Figure 5, 3PB typically finds a better UB after spending 60% of the overall time, whereas 2LBB requires 73%. When a better solution is found, a large number of nodes can be pruned, accelerating the remaining solution process. We believe this explains why the

time reduction diminishes. Another interesting observation is that 2LBB-dy explores 7% fewer nodes than 2LBB, which is the opposite of what we observed in Table 3. This is because, when given a larger UB, the tree size is larger, making BKF tend to run more LP testing to achieve the best possible tradeoff between testing efforts and tree size.

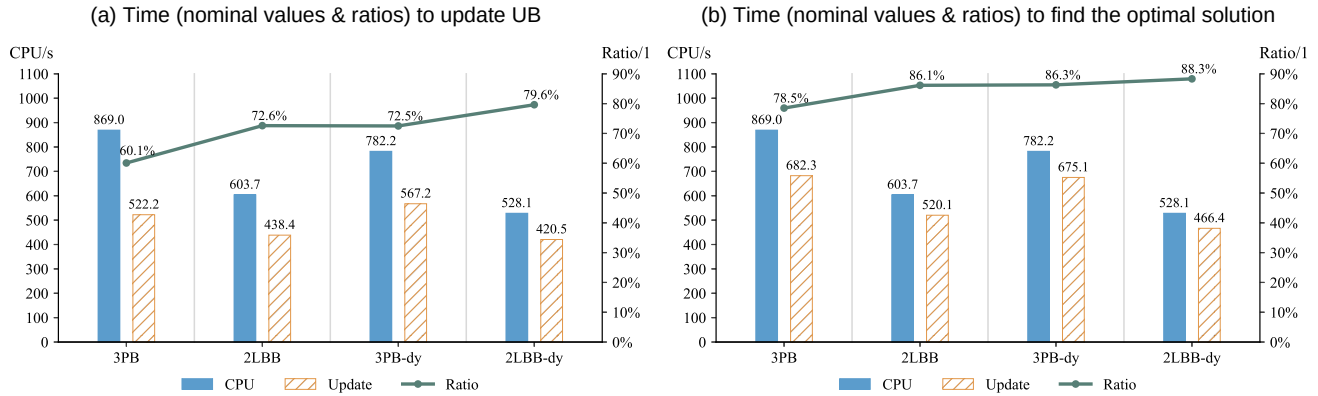
6.1.2. Generalization Capability of the 2LBB. Generalization refers to the capability of a trained ML model to effectively adapt to new, previously unseen data drawn from the same distribution as the one used during model training. Generalization affects the practicability of a trained model. We investigate the generalization of our 2LBB by evaluating its performance in solving instances of different sizes. More specifically, we collect the training data by solving instance batch CVRP-150-270-0 and apply the resulting 2LBB to solve batches CVRP-120-90-1, CVRP-150-90-1, and CVRP-180-90-1. As illustrated by the ratios of tree size and CPU time with respect to the 3PB in Figure 6, the effectiveness of the 2LBB approach is consistent across instances with 120, 150, and 180 customers. These results confirm that our methodology generalizes well. Especially, the node ratio decreases as the size increases, indicating that 2LBB is potentially more powerful in solving larger instances.

6.1.3. Performance on Established CVRP Benchmark Instances. In this section, we report the computational results for the established CVRP benchmark instances, that is, batch CVRP-V-42-X. Most of these instances remain very challenging to prove optimality within several days, even with the best-known value as the initial UB. Some instances may still remain unsolved. Only reporting the results of instances that can be solved within a few hours is not enough to draw rigorous conclusions. Therefore, to include as many instances as possible, we decided to solve the more difficult instances (34 of 42 instances) by setting

Table 4. Comparison of 3PB, 2LBB, 3PB-dy, and 2LBB-dy When u^0 Serves as the UB

Version	Root/s	Root gap/%	LP/s	Heuristic/s	CPU/s	No. of nodes
3PB	24.2	0.24	356.3	238.1	869.0	463.8
2LBB			317.8	71.9	603.7	398.3
3PB-dy			390.2	181.3	782.2	347.6
2LBB-dy			269.2	57.6	528.1	372.1

Figure 5. (Color online) Comparison on Time (Nominal Values and Ratios) for Updating the UB and Finding the Optimal Solution



an initial *cutoff* that is geometrically 0.14% lower than the best-known value, allowing the solver to stop within approximately two hours. It is important to note that the specified cutoff might not always represent a valid UB. The solver could terminate without finding any feasible solution. In this case, we conclude the given initial cutoff is a valid lower bound (LB). However, if at least one feasible solution is found, then the output solution is guaranteed to be optimal. For clarity, we consistently use the term *solve* to encompass both outcomes.

The key reason for not comparing the LB within the same time limit, as often done in other papers, is that BKF is not designed to maximize the LB under tight running time constraints but rather focuses on minimizing the total solution time, even though this necessitates spending significantly more time at the early stages of branching. In the dynamic version, fewer nodes are explored if the solver stops prematurely, potentially resulting in a worse LB and leading to the incorrect conclusion that the dynamic version underperforms. However, the dynamic version may significantly outperform the static version in later stages, because the initial intensive efforts lead to fewer

nodes needing to be solved, each also requiring less testing. Ultimately, the dynamic version requires less time for the complete solution process. Additionally, it is both inconsistent and challenging to estimate total time reduction by comparing LB improvements.

Table 5 shows that 2LBB-dy still achieves the best performance among all four versions of RouteOpt, specifically achieving a 15% time reduction compared with 2LBB and extending to a 38% time reduction for 3PB. Additionally, because cutting planes are enabled, which increases the average time excluding LP testing for solving a BBN, the dynamic version opts for more LP testing, which results in a node reduction of 31% and 16% for 3PB-dy versus 3PB and 2LBB-dy versus 2LBB, respectively. What is noteworthy is that 3PB-dy achieves a 28% time reduction compared with 3PB, demonstrating again the potential of the generalized and independent learning mechanism of BKF, which can significantly enhance its impact in learning-free branching strategies including the 3PB.

From the detailed results in Section EC.10.3 of the e-companion, we observed that for instances requiring fewer branchings—specifically, fewer than 64 using 2LBB—the time reduction of 2LBB-dy compared with

Figure 6. (Color online) Generalization Capability of 2LBB Across Different Instance Sizes

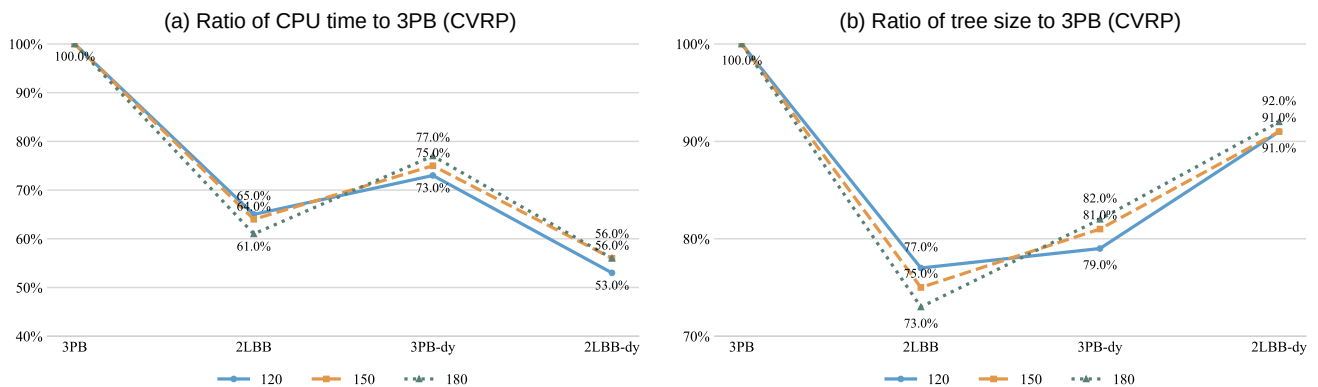


Table 5. Comparison of 3PB, 2LBB, 3PB-dy, and 2LBB-dy on Batch CVRP-V-42-X

Version	Root/s	Root gap/%	LP/s	Heuristic/s	CPU/s	No. of nodes
3PB	366.8	0.11	389.2	329.2	4,324.3	100.3
2LBB			321.9	91.2	3,184.3	74.9
3PB-dy			410.6	256.4	3,120.7	69.3
2LBB-dy			329.0	86.1	2,691.5	62.7

2LBB and 3PB is 9% and 32%, respectively. Conversely, for instances with more than 64 branchings, we observe time reductions of 22% and 43%, respectively. This larger time reduction for instances requiring extensive branching indicates that when more branchings are needed, a static $\hat{\theta}$ is less effective. Consequently, the application of BKF in solving more difficult instances gives rise to even more pronounced acceleration.

We also noticed that the time reduction brought by BKF decreases for CVRP as the average route length increases. Specifically, when using 8 as the threshold to categorize instances into two groups—short routes (<8) and long routes (≥ 8)—we observed that for the short group, the time reduction of 2LBB-dy compared with 2LBB is 22%, whereas it significantly decreases to 10% for the long group. A similar observation can be made for 3PB-dy. The reason is that instances with longer routes incur significantly higher pricing time for each node, resulting in a smaller ratio of branching selection time versus overall time. Even though BKF helps greatly reduce computational time for later branching selection, the overall time reduction remains relatively minor. Additionally, the long group instances have smaller BBT sizes than the short group, which further exacerbates this effect.

6.1.4. Comparison with the VRPSolver. The VRPSolver, developed in Pessoa et al. (2020), is recognized as the best-performing tool for solving the CVRP exactly. We compare its performance with the RouteOpt version equipped with 2LBB-dy, which achieves the best results among the four tested versions. For testing, we select the CVRP-V-42-X instance batch. To ensure a fair comparison, we use the same cutoff, the same LP and MIP solver, CPLEX (version 22.1.0 in our experiments; IBM Corporation 2023) with identical parameters. VRPSolver with extension version v0.5.24 and BaPCod version v0.74, both using default parameters, are employed to solve the instances.

From Table 6, we observe that 2LBB-dy processes the root node much faster, largely because of RouteOpt's less aggressive cutting plane addition compared with VRPSolver, which we refer to as the branching-focused strategy, in contrast to VRPSolver's cutting-focused strategy. We adopt the branching-focused strategy for two main reasons. First, in 2LBB-dy, good branching decisions can be made without excessive computational efforts, making it more efficient to focus on branching rather than performing numerous rounds of cutting plane addition that offer minimal improvement to the lower bound. Second, RouteOpt's current heuristics for separating rank-1 cuts are not as effective as VRPSolver's. RouteOpt typically has a larger root gap, approximately 0.03% larger in this case.

Despite such a larger root gap, RouteOpt still reduces the overall solution time by 47% compared with VRPSolver. RouteOpt requires significantly less time at each node for both the pricing and cutting phases thanks to the less aggressive cutting strategy and the 2LBB-dy scheme that efficiently makes good branching decisions. This substantial time reduction per node compensates for the significantly increased number of nodes explored. We emphasize that this acceleration is not simply the result of a direct switch to a branching-focused strategy. Rather, it is the effectiveness brought by 2LBB-dy that ensures each branching decision requires less computational effort without little sacrifice in quality, ultimately leading to the success of our RouteOpt.

6.2. Case Study II: VRPTW

We further demonstrate the effectiveness of 2LBB and BKF by applying them to various VRPTW instance batches: VRPTW-180-60-1, VRPTW-160-60-1, VRPTW-200-60-1, and VRPTW-200-22-H2. The critical parameters used, including those for the dynamic version (α and η), are consistent with those in the CVRP case study, with the exception of VRPTW-200-22-H2. For

Table 6. Comparison of VRPSolver and 2LBB-dy on Batch CVRP-V-42-X

Version	Root/s	Root gap/%	LP/s	Heuristic/s	CPU/s	No. of nodes
VRPSolver	1,178.8	0.08	336.1	116.4	5,773.3	21.8
2LBB-dy	429.1	0.11	316.6	146.2	3,055.1	64.0

Table 7. Comparison of 3PB, 2LBB, 3PB-dy, and 2LBB-dy with u^* Serving as the UB

Version	Root/s	Root gap/%	LP/s	Heuristic/s	CPU/s	No. of nodes
3PB	24.0	0.17	175.3	124.0	414.0	176.0
2LBB			141.9	24.1	254.1	138.8
3PB-dy			119.2	92.2	317.0	157.6
2LBB-dy			79.1	17.1	201.5	164.5

this instance batch, we observed a decay in the effectiveness of $M1$ and selected a larger $\alpha = 0.45$ to address it. Additionally, we chose a smaller $\eta = 0.5$ in response to the decreased testing scores observed as branching progresses. All these instances are tested under Setting II, even for VRPTW-200-22-H2, because most of them require very few branchings or are very hard to solve even after hours of effort using Setting I. Therefore, no comparison with VRPSolver is made in this case study, because we do not enable RouteOpt to utilize all of its techniques. Readers can refer to Section EC.9.2 of the e-companion for training details.

6.2.1. Superior of the 2LBB. In this section, we compare the performance of four versions of RouteOpt for instance batch VRPTW-180-60-1 and report the same metrics used in the CVRP case study. Detailed results for each instance are provided in Sections EC.10.5 and 10.6 of the e-companion.

As shown in Table 7, the results are consistent with those from the CVRP case study. Specifically, the time reduction for 2LBB compared with 3PB is 39%, and for 2LBB-dy compared with 3PB-dy, it is 36%. 2LBB-dy exhibits the best performance, achieving a 51% time reduction compared with 3PB. Furthermore, by incorporating BKF, 3PB-dy reduces computational time by 23% compared with 3PB. This once again demonstrates the effectiveness of BKF over a static scheme, even in a learning-free branching strategy.

From Table 8, a similar reduction in time reduction is observed. Specifically, the time reduction of 2LBB compared with 3PB diminishes to 34%, and for 2LBB-dy compared with 3PB, the reduction is 48%. As previously explained in the CVRP case study, this diminishing effect can be attributed to the relatively early updating of the UB or the early finding of the optimal solution, as illustrated in Figure 7. Moreover, because the UB is larger, BKF tends to prioritize decreasing the tree size. Consequently, the node ratios for both

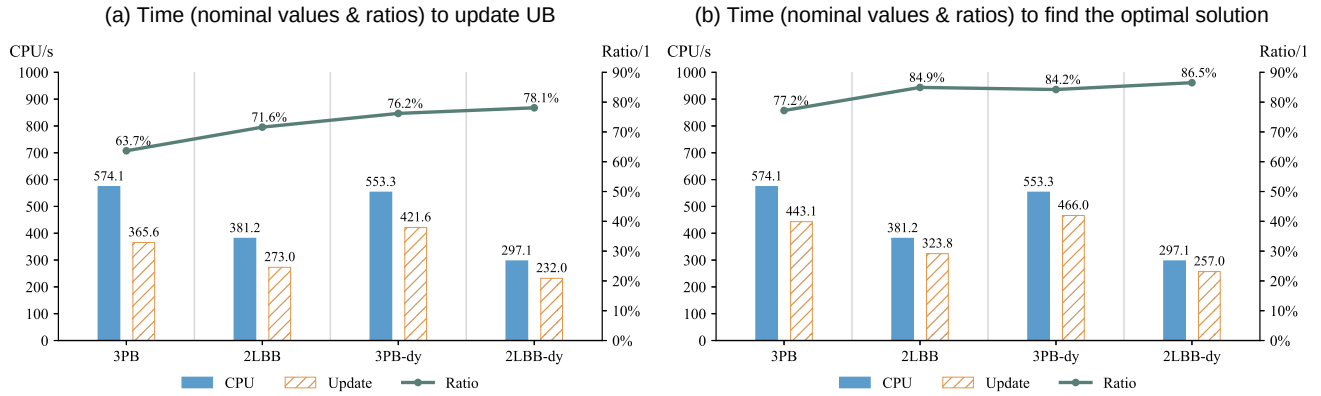
3PB-dy versus 3PB and 2LBB-dy versus 2LBB have decreased compared with the results when the optimal value u^* is used as the UB.

6.2.2. Generalization Capability of the 2LBB. In this section, we collect training data by solving instance batch VRPTW-180-600-0 and subsequently apply the trained $M1$ and $M2$ to solve instance batches VRPTW-160-60-1, VRPTW-180-60-1, and VRPTW-200-60-1. As illustrated by the ratios of tree size and CPU time with respect to 3PB in Figure 8, the effectiveness of the 2LBB approach is consistent across instances with 160, 180, and 200 customers, achieving an approximate 40% time reduction. This reduction increases to 50% after incorporating BKF. Even without introducing the learning models to RouteOpt, applying just BKF to 3PB results in a 10%–26% time reduction, with the reduction increasing as the instance size grows. This again indicates that BKF becomes even more effective when dealing with larger instances.

6.2.3. Performance on Established VRPTW Benchmark Instances. We report the computational results for the established VRPTW benchmark instances: batch VRPTW-200-22-H2. Our preliminary results suggest that these instances tend to favor an aggressive cutting strategy that achieves a very small root gap and thus requires minimal need for branching. This characteristic, unfortunately, is not ideal for demonstrating the effectiveness of our learning-to-branch scheme. To increase the reliance on branching decisions, we continue employing Setting II in this set of experiments. Furthermore, we stop adding cuts when the current gap between the LB and the cutoff is within 0.50%. This adjustment necessitates significantly more branching decisions, thereby suitable for demonstrating the differences among the four branching strategies. The cutoff is chosen to be 0.64% (on a geometric mean basis) smaller than the optimal value

Table 8. Comparison of 3PB, 2LBB, 3PB-dy, and 2LBB-dy When u^0 Serving as the UB

Version	Root/s	Root gap/%	LP/s	Heuristic/s	CPU/s	No. of nodes
3PB	21.8	0.28	258.0	187.8	574.1	245.9
2LBB			237.3	52.0	381.2	218.1
3PB-dy			269.4	164.9	553.3	220.1
2LBB-dy			158.0	39.8	297.1	219.9

Figure 7. (Color online) Comparison on Time (Nominal Values and Ratios) for Updating the UB and Finding the Optimal Solution

for 8 of the 22 instances to ensure the solver stops within a four-hour limit when using 2LBB-dy. For the remaining 14 relatively easy instances, the optimal value serves as the cutoff.

As shown in Table 9, instance batch VRPTW-200-22-H2 exhibits significantly different characteristics compared with other instances. Specifically, for the 3PB version, where each branching decision performs LP testing for 10 candidates and heuristic testing for 2 candidates, the time per LP testing calculates to approximately $\frac{253.4}{10b} \approx \frac{25.3}{b}$ seconds, and the time per heuristic test is $\frac{1284.8}{2b} \approx \frac{642.4}{b}$ seconds, where b is the number of branchings. This indicates that heuristic testing is more than 25 times as time-consuming as LP testing, suggesting that applying BKF to the heuristic phase could offer substantial time reduction. After incorporating BKF into the heuristic phase as described in Section EC.6.1.2 of the e-companion, the dynamic versions show significant improvements. Comparing 3PB-dy to 3PB, a time reduction of 29% and a reduction in the number of nodes by 32% were achieved. Similarly, when comparing 2LBB-dy with 2LBB, a time reduction of 26% and a reduction in the number of nodes by 44% were

achieved. Furthermore, when comparing 2LBB-dy to 3PB, a CPU time reduction of 51% was achieved.

6.3. Open CVRP Instances

In this section, we investigate the performance of RoutOpt across four versions to tackle open instances for which optimal solutions are unknown. We picked five instances from the CVRPLIB X series: X-n294-k50, X-n313-k71, X-n336-k84, X-n344-k43, and X-n384-k52. These instances seem relatively easier to solve because they possess short average route lengths. While preparing this paper, we became aware that X-n344-k43 was solved optimally on September 14, 2023, by the VRPSolver with the cluster branching strategy in Silva and Uchoa (2022). Because no details about their results were reported, the computational effort they spent to solve it is unclear. We decided to still report the relevant results for completeness. It is worth mentioning that using one thread can take an excessive amount of time (several months) to complete the computation, which necessitates taking advantage of the rich computational resources we can access.

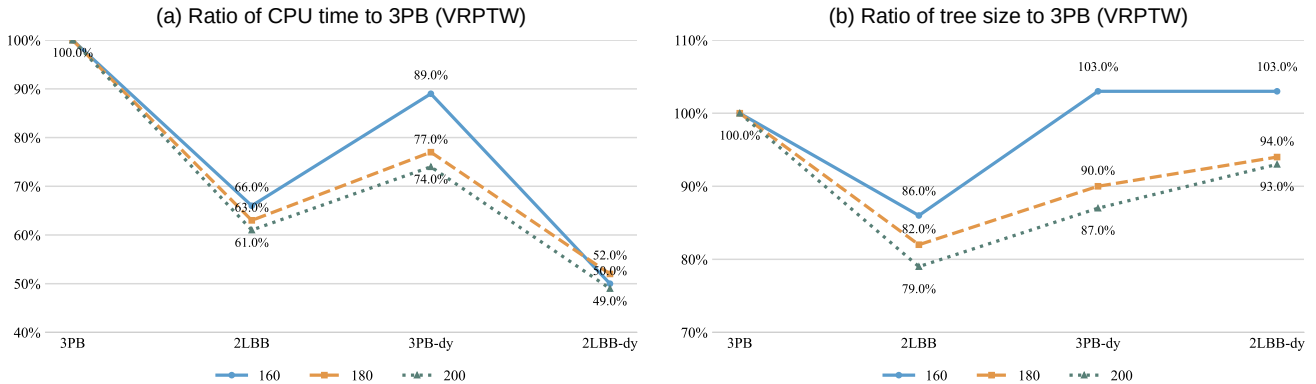
Figure 8. (Color online) Generalization Capability of 2LBB Across Different Instance Sizes

Table 9. Comparison of 3PB, 2LBB, 3PB-dy, and 2LBB-dy on Batch VRPTW-200-22-H2

Version	Root/s	Root gap/%	LP/s	Heuristic/s	CPU/s	No. of nodes
3PB	169.5	0.38	253.4	1,284.8	2,959.2	114.7
2LBB			245.4	324.7	1,944.8	122.7
3PB-dy			253.7	743.9	2,112.5	78.2
2LBB-dy			147.8	351.8	1,442.6	68.6

More specifically, we leverage the supercomputing resources on HiPerGator following the two-step procedure elaborated below. In the solution process, enumeration starts to succeed at some BBNs when the gap becomes small, and such BBNs are referred to as *enumerated BBNs*. We write the information of each enumerated BBN into a separate file and prune it from the BBT. This process continues until there are no open BBNs in the BBT and is referred to as the first step. This step is much more time-consuming, taking around a week because no parallelism is implemented. In the second step, each enumerated BBN is solved as a standalone job using one thread and 16 GB of memory, which usually finishes within three hours. HiPerGator enables us to process 500 jobs simultaneously, and the second step typically takes less than a day.

The 3PB is configured to handle such extremely difficult instances with $\hat{\theta}_{pf} = 30$ and $\hat{\theta} = 3$. For a fair comparison, $\hat{\theta}_{pf} = 100$, $\hat{\theta} = 30$, and $\hat{\theta} = 3$ are used for 2LBB. We managed to prove optimality for two instances: X-n294-k50 and X-n344-k43. Table 10 lists the time for the root node (Root/s), the lower bound for the root node (LB), and the upper bound as input (UB). In addition to the same time metrics, we add one more column indicating the number of enumerated BBNs (No. of E-nodes).

We observe that the 2LBB consistently outperforms the 3PB for both instances. Specifically, for X-n294-k50, the time reduction is 21%, and for X-n344-k43, the time reduction is 36%. As expected, both dynamic versions achieve better performance than their static counterparts, especially in the comparison of 3PB-dy and 3PB for instance X-n294-k50. Although 3PB devotes much more time to branching selection, the number of nodes

is 1.4 times that of 3PB-dy, showing the effectiveness of the dynamic version. It is worth noting that, for instance X-n294-k50, 3PB-dy achieves better performance, a time reduction of 8%, than 2LBB-dy. Considering that the difference in time spent on LP testing is minimal, the remaining factor could be inaccuracies in the second stage selection procedure. We remark that in this experiment, the selection procedure uses the same $\underline{d}$ and δ throughout the paper, which may not be an ideal choice for instances with more than 20,000 BBNs. To elaborate, a large $\underline{d}$ and a small δ may cause the second stage to skip necessary testing, leading to suboptimal branching decisions. This negative impact can be amplified for instances with a large BBT size. It also suggests that a dynamic version adjusting this parameter should be beneficial.

7. Concluding Remarks

We propose the first effective learning framework that significantly enhances the SOTA 3PB strategy for exact BPC algorithms, along with a novel theoretical model that establishes the foundations for balancing branching decision time and quality. The 2LBB-dy, derived from this framework, demonstrates exceptional performance, achieving approximately 45% and 50% time reduction over the SOTA 3PB for CVRP and VRPTW, respectively, and enabling our RouteOpt to significantly outperform the best existing competitor, VRPSolver. Through a comprehensive numerical study, we further demonstrate that the learned strategy generalizes well to large unseen problems with similar structures and thus is especially suitable for scenarios where similar problems need to be solved on a regular basis, such as last-mile delivery in a given region.

Table 10. Comparison of 3PB, 2LBB, 3PB-dy, and 2LBB-dy on Instances X-n294-k50 and X-n344-k43

Instance	Version	Root/s	LB	UB	LP/h	Heuristic/h	CPU/h	No. of nodes	No. of E-nodes
X-n294-k50	3PB	79	46,928	47,161	28.3	25.4	170.5	23,355	11,678
	2LBB				24.0	10.0	135.1	20,123	10,062
	3PB-dy				9.0	15.7	111.6	16,353	8,176
	2LBB-dy				9.4	8.3	121.6	21,297	10,649
X-n344-k43	3PB	172	41,922	42,050	1.5	2.6	13.7	891	444
	2LBB				1.2	0.6	8.8	731	363
	3PB-dy				1.0	1.5	9.6	707	349
	2LBB-dy				0.7	0.5	7.8	687	342

2LBB-dy can be readily extended to problems beyond VRP, such as job scheduling and network optimization, especially those depending on column generation. Although the specific features used to train the ML models may differ across problem domains, a key insight from this work is the importance of focusing on features that capture the problem's intrinsic structure, such as the distribution of node locations and demands in the CVRP, rather than solely relying on the nominal mathematical formulation. That said, the effectiveness of 2LBB depends on the predictive accuracy of $M1$ and $M2$. In more complex problems than CVRP or VRPTW, such as inventory routing problems (Engineer et al. 2012, Desaulniers et al. 2016), healthcare planning problems (Trautsumwieser and Hirsch 2014, Qiu et al. 2022), and crew rostering problems (Quesnel et al. 2020, Breugem et al. 2022), hierarchical branching is often employed, involving multiple types of branching variables. Using a single model to rank candidates across all variable types may reduce prediction accuracy and degrade 2LBB-dy's performance. A more practical solution is to train separate models for each variable type, rank candidates within their respective groups, and then use an additional stage to select the overall best candidate for branching.

Another prospective research direction concerns adapting the current method to other branching variable types or multiple variables at a time (Yang et al. 2021). For example, the recently proposed clustering branching (Silva and Uchoa 2022) has helped to solve over 20 open instances, suggesting its potential to tackle instances with certain structures. Nevertheless, the clustering branching gives rise to an exponentially large number of branching candidates (possible clusters), posing an evident challenge in the learning process. Moreover, incorporating new features characterizing such cluster structures is anticipated to be nontrivial. Finally, despite the wide applicability of our theoretical model, some branching variable types might deviate far from Assumption (g), suggesting a need for alternative assumptions for such cases. Most ideas proposed in deriving the BKF are expected to still apply therein.

Acknowledgments

The authors thank the area editor, associate editor, and three anonymous referees for providing valuable comments and suggestions.

References

- Achterberg T (2007) Constraint integer programming. PhD thesis, Technische Universität, Berlin.
- Achterberg T, Wunderling R (2013) Mixed integer programming: Analyzing 12 years of progress. *Facets of Combinatorial Optimization* (Springer, Berlin), 449–481.
- Alvarez AM, Louveaux Q, Wehenkel L (2017) A machine learning-based approximation of strong branching. *INFORMS J. Comput.* 29(1):185–195.
- Applegate D, Bixby R, Chvátal V, Cook W (1995) Finding cuts in the tsp (a preliminary report). Technical report, Center for Research in Parallel Computing, Rice University, Houston.
- Augerat P, Naddef D, Belenguer J, Benavent E, Corberan A, Rinaldi G (1995) Computational results with a branch and cut code for the capacitated vehicle routing problem. Technical Report No. 949-M, Université Joseph Fourier, Grenoble, France.
- Baldacci R, Christofides N, Mingozzi A (2008) An exact algorithm for the vehicle routing problem based on the set partitioning formulation with additional cuts. *Math. Programming* 115(2):351–385.
- Baldacci R, Hadjiconstantinou E, Mingozzi A (2004) An exact algorithm for the capacitated vehicle routing problem based on a two-commodity network flow formulation. *Oper. Res.* 52(5):723–738.
- Baldacci R, Mingozzi A, Roberti R (2011) New route relaxation and pricing strategies for the vehicle routing problem. *Oper. Res.* 59(5):1269–1283.
- Bektaş T, Erdoğan G, Røpke S (2011) Formulations and branch-and-cut algorithms for the generalized vehicle routing problem. *Transportation Sci.* 45(3):299–316.
- Bénichou M, Gauthier JM, Girodet P, Hentges G, Ribière G, Vincent O (1971) Experiments in mixed-integer linear programming. *Math. Programming* 1(1):76–94.
- Breugem T, Dollevoet T, Huisman D (2022) Is equality always desirable? Analyzing the trade-off between fairness and attractiveness in crew rostering. *Management Sci.* 68(4):2619–2641.
- Cappart Q, Chételat D, Khalil EB, Lodi A, Morris C, Veličković P (2023) Combinatorial optimization and reasoning with graph neural networks. *J. Machine Learn. Res.* 24(130):1–61.
- Contardo C, Martinelli R (2014) A new exact algorithm for the multi-depot vehicle routing problem under capacity and route length constraints. *Discrete Optim.* 12:129–146.
- Costa L, Contardo C, Desaulniers G (2019) Exact branch-price-and-cut algorithms for vehicle routing. *Transportation Sci.* 53(4):946–985.
- Costa PR, Rhuggenaath J, Zhang Y, Akcay A (2020) Learning 2-opt heuristics for the traveling salesman problem via deep reinforcement learning. Pan SJ, Sugiyama M, eds. *Proc. 12th Asian Conf. Machine Learn.* vol. 129 (PMLR, New York), 465–480.
- Dantzig GB, Ramser JH (1959) The truck dispatching problem. *Management Sci.* 6(1):80–91.
- Desaulniers G, Rakke JG, Coelho LC (2016) A branch-price-and-cut algorithm for the inventory-routing problem. *Transportation Sci.* 50(3):1060–1076.
- Desrosiers J, Soumis F, Desrochers M (1984) Routing with time windows by column generation. *Networks* 14(4):545–565.
- Engineer FG, Furman KC, Nemhauser GL, Savelsbergh MW, Song JH (2012) A branch-price-and-cut algorithm for single-product maritime inventory routing. *Oper. Res.* 60(1):106–122.
- Fukasawa R, Longo H, Lysgaard J, Aragão M, Reis M, Uchoa E, Werneck RF (2006) Robust branch-and-cut-and-price for the capacitated vehicle routing problem. *Math. Programming* 106(3):491–511.
- Gasse M, Chételat D, Ferroni N, Charlin L, Lodi A (2019) Exact combinatorial optimization with graph convolutional neural networks. Wallach H, Larochelle H, Beygelzimer A, d'Alché-Buc F, Fox E, Garnett R, eds. *Adv. Neural Inform. Processing Systems* (Curran Associates Inc., Red Hook, NY), 15554–15566.
- Gurobi Optimization, LLC (2023) Gurobi optimizer reference manual. Accessed February 13, 2026, <https://www.gurobi.com/documentation/10.0/refman/index.html>.
- Heching A, Hooker JN, Kimura R (2019) A logic-based benders approach to home healthcare delivery. *Transportation Sci.* 53(2):510–522.
- Homberger J, Gehring H (2005) A two-phase hybrid metaheuristic for the vehicle routing problem with time windows. *Eur. J. Oper. Res.* 162(1):220–238.

- IBM Corporation (2023) IBM ILOG CPLEX optimization studio CPLEX user's manual. <https://www.ibm.com/docs/en/icos/22.1.0?topic=manuals-cplex-users-manual>.
- Irnich S, Desaulniers G, Desrosiers J, Hadjar A (2010) Path-reduced costs for eliminating arcs in routing and scheduling. *INFORMS J. Comput.* 22(2):297–313.
- Jepsen M, Petersen B, Spoorendonk S, Pisinger D (2008) Subset-row inequalities applied to the vehicle-routing problem with time windows. *Oper. Res.* 56(2):497–511.
- Khalil E, Dai H, Zhang Y, Dilkina B, Song L (2017) Learning combinatorial optimization algorithms over graphs. Guyon I, Von Luxburg U, Bengio S, Wallach H, Fergus R, Vishwanathan SVN, Garnett R, eds. *Adv. Neural Inform. Processing Systems*, vol. 30, (Curran Associates, Inc., Red Hook, NY), 6348–6358.
- Khalil E, Le Bodic P, Song L, Nemhauser G, Dilkina B (2016) Learning to branch in mixed integer programming. *Proc. AAAI Conf. Artificial Intelligence* 30.
- Laporte G (2009) Fifty years of vehicle routing. *Transportation Sci.* 43(4):408–416.
- Laporte G, Nobert Y (1983) A branch and bound algorithm for the capacitated vehicle routing problem. *Oper. Res. Spektrum* 5(2):77–85.
- Le Bodic P, Nemhauser G (2017) An abstract model for branching and its application to mixed integer programming. *Math. Programming* 166(1–2):369–405.
- Lima I, Uchoa E, Pecin D, Pessoa A, Poggi M, Vidal T, Subramanian A, et al. (2014) CVRPLIB: Capacitated vehicle routing problem library. <http://vrp.galagos.inf.puc-rio.br/index.php/en/updates>.
- Linderroth JT, Savelsbergh MW (1999) A computational study of search strategies for mixed integer programming. *INFORMS J. Comput.* 11(2):173–187.
- Lübbecke ME, Desrosiers J (2005) Selected topics in column generation. *Oper. Res.* 53(6):1007–1023.
- Lysgaard J, Letchford AN, Eglese RW (2004) A new branch-and-cut algorithm for the capacitated vehicle routing problem. *Math. Programming* 100(2):423–445.
- Mai TL, Navet N (2021) Deep learning to predict the feasibility of priority-based ethernet network configurations. *ACM Trans. Cyber-Physical Systems* 5(4):1–26.
- Morabit M, Desaulniers G, Lodi A (2021) Machine-learning-based column selection for column generation. *Transportation Sci.* 55(4):815–831.
- Morabit M, Desaulniers G, Lodi A (2022) Machine-learning-based arc selection for constrained shortest path problems in column generation. *INFORMS J. Optim.* 5(2):191–210.
- Naddef D, Rinaldi G (2002) Branch-and-cut algorithms for the capacitated VRP. Toth P, Vigo D, eds. *The Vehicle Routing Problem* (SIAM, Philadelphia), 53–84.
- Nazari M, Oroojlooy A, Snyder L, Takác M (2018) Reinforcement learning for solving the vehicle routing problem. Bengio S, Wallach H, Larochelle H, Grauman K, Cesa-Bianchi N, Garnett R, eds. *Advances in Neural Information Processing Systems*, vol. 31 (Curran Associates Inc., Red Hook, NY), 9839–9849.
- Pecin D, Contardo C, Desaulniers G, Uchoa E (2017a) New enhancements for the exact solution of the vehicle routing problem with time windows. *INFORMS J. Comput.* 29(3):489–502.
- Pecin D, Pessoa A, Poggi M, Uchoa E (2017b) Improved branch-cut-and-price for capacitated vehicle routing. *Math. Programming Comput.* 9(1):61–100.
- Pecin D, Pessoa A, Poggi M, Uchoa E, Santos H (2017c) Limited memory rank-1 cuts for vehicle routing problems. *Oper. Res. Lett.* 45(3):206–209.
- Pereira P, Courtade E, Aloise D, Quesnel F, Soumis F, Yaakoubi Y (2022) Learning to branch for the crew pairing problem. Les Cahiers du GERAD. Technical Report, GERAD, HEC Montréal.
- Pessoa A, Sadykov R, Uchoa E, Vanderbeck F (2020) A generic exact solver for vehicle routing and related problems. *Math. Programming* 183(1):483–523.
- Qiu H, Wang D, Yin Y, Cheng TE, Wang Y (2022) An exact solution method for home health care scheduling with synchronized services. *Naval Res. Logist.* 69(5):715–733.
- Quesnel F, Desaulniers G, Soumis F (2020) Improving air crew rostering by considering crew preferences in the crew pairing problem. *Transportation Sci.* 54(1):97–114.
- Silva JMP, Uchoa E (2024) Cluster branching for vehicle routing problems. *INFORMS J. Comput.* 36(6):1774–1791.
- Tahir A, Quesnel F, Desaulniers G, El Hallaoui I, Yaakoubi Y (2021) An improved integral column generation algorithm using machine learning for aircrew pairing. *Transportation Sci.* 55(6):1411–1429.
- Trautsamwieser A, Hirsch P (2014) A branch-price-and-cut approach for solving the medium-term home health care planning problem. *Networks* 64(3):143–159.
- Uchoa E, Pecin D, Pessoa A, Poggi M, Vidal T, Subramanian A (2017) New benchmark instances for the capacitated vehicle routing problem. *Eur. J. Oper. Res.* 257(3):845–858.
- van der Hagen L, Agatz N, Spliet R, Visser TR, Kok L (2024) Machine learning-based feasibility checks for dynamic time slot management. *Transportation Sci.* 58(1):94–109.
- Vinsensius A, Wang Y, Chew EP, Lee LH (2020) Dynamic incentive mechanism for delivery slot management in e-commerce attended home delivery. *Transportation Sci.* 54(3):567–587.
- Yang Y, Boland N, Savelsbergh M (2021) Multivariable branching: A 0-1 knapsack problem case study. *INFORMS J. Comput.* 33(4):1354–1367.
- Yang Y, Boland N, Dilkina B, Savelsbergh M (2022) Learning generalized strong branching for set covering, set packing, and 0-1 knapsack problems. *Eur. J. Oper. Res.* 301(3):828–840.
- Yang Y, Yan C, Cao Y, Roberti R (2023) Planning robust drone-truck delivery routes under road traffic uncertainty. *Eur. J. Oper. Res.* 309(3):1145–1160.
- Zhang X, Chen L, Gendreau M, Langevin A (2022) Learning-based branch-and-price algorithms for the vehicle routing problem with time windows and two-dimensional loading constraints. *INFORMS J. Comput.* 34(3):1419–1436.
- Zheng YJ (2018) Emergency train scheduling on Chinese high-speed railways. *Transportation Sci.* 52(5):1077–1091.

Zhengzhong You is an independent researcher. His research interests include branch-price-and-cut algorithms, column generation, and learning-augmented methods for large-scale integer and combinatorial optimization, with applications to exact vehicle routing. He is a lead developer of RouteOpt, an open-source, modular exact solver for vehicle routing problems.

Yu Yang is an assistant professor in the department of industrial and systems engineering at the University of Florida (UF) and assistant director of the UF Center for Applied Optimization. His research focuses on machine learning and artificial intelligence for combinatorial optimization, large-scale optimization theory, and their applications in supply chain management and sustainable energy system design. His team developed RouteOpt.

Xinshang Wang is a researcher at Alibaba Group (U.S.). His research focuses on large-scale integer and stochastic optimization, with an emphasis on learning-augmented algorithms for revenue management, assortment and pricing, and operational decision making in logistics and supply chain systems.

Wotao Yin is an applied mathematician, scientist, and engineer and is the director of the Decision Intelligence Lab at the Alibaba DAMO Academy. His research includes computational optimization, operator splitting methods, and their applications in image processing and machine learning. He has received many prestigious awards, including the Morningside Gold Medal, the NSF CAREER Award, the Alfred P. Sloan Research Fellowship, and the INFORMS Egon Balas Prize.